מיני פרויקט בבסיסי נתונים

אגף חיל השריון של צה"ל

מגישים: יניב טל - 216196550

אהוביה בצלאל - 329228431

מרצה : מר יעקב ברזילי

קישור לgithub:

https://github.com/yaniv-tal/DBProject_216196550_329228431

תוכן עניינים

שלב 1 - עיצוב ויצירת DB, שימוש בERDPLUS והכנסת נתונים 4
תיאור המערכת 4
תרשים ERD של הישויות והקשרים... 5
תרשים DSD.. 6
תיאור הישויות והתכונות שלהן.. 6
תיאור הקשרים... 7
רשימת הסכמות של בסיס הנתונים... 7
הסכמות של הישויות 7
הסכמות של הקשרים 7
הוכחה שהסכמות מנורמלות ב3NF... 8
קוד SQL של יצירת הטבלאות (CREATE) 8
קוד SQL של זריקת הטבלאות (DROP) 10
הכנסת מידע לטבלאות.. 10
הכנסת מידע לSOLIDERS – דרך 1, Mockaroo file 10
הכנסת מידע לCOMMANDER – דרך 2, קובץ Excel... 12
הכנסת מידע לCREWMATE – דרך 3, סקריפט בפייתון 14
הכנסת מידע לUNIT – דרך 4, קובץ טקסט.. 15
הכנסת מידע לTANK – דרך 3, סקריפט בפייתון 17
הכנסת מידע לMISSION – דרך 1, Mockaroo file 18
הכנסת מידע לparticipate – דרך 2, קובץ Excel.. 21
גיבוי הטבלאות 23
שחזור הטבלאות.. 23
שלב 2 - שאילתות (וכן השאילתות עם פרמטרים) 26
שאילתות.. 26
8 שאילתות select:... 26
3 שאילתות delete:... 34
3 שאילתות update: 38
שאילתת Update עם Commit:.. 41
אילוצים.. 43
שלב 3 – אינטגרציה ומבטים... 46
עיצוב DSD וERD חדשים 46
עיצוב DSD של הקבוצה השנייה.. 46
עיצוב ERD של הקבוצה השנייה 46
עיצובי ERD ו- DSD משולבים של שתי הקבוצות... 47
עדכון בסיס הנתונים 49

הסקת הסכמות החדשות מתוך הERD המשולב..49
הכנסת נתונים לטבלאות 51
מבטים..53
יצירת 2 מבטים..53
יצירת 2 שאילתות עבור כל מבט...56
שלב 4 – תכנות 61
פונקציות 61
פרוצדורות 64
תוכניות ראשיות...67
טריגרים..71
שלב 5 – ממשק גרפי 74
כלים...74

מהלך העבודה...75

הוראות הפעלה של הממשק + צילומי מסך..76

שלב 1 - עיצוב ויצירת DB, שימוש בERDPLUS והכנסת נתונים

תיאור המערכת

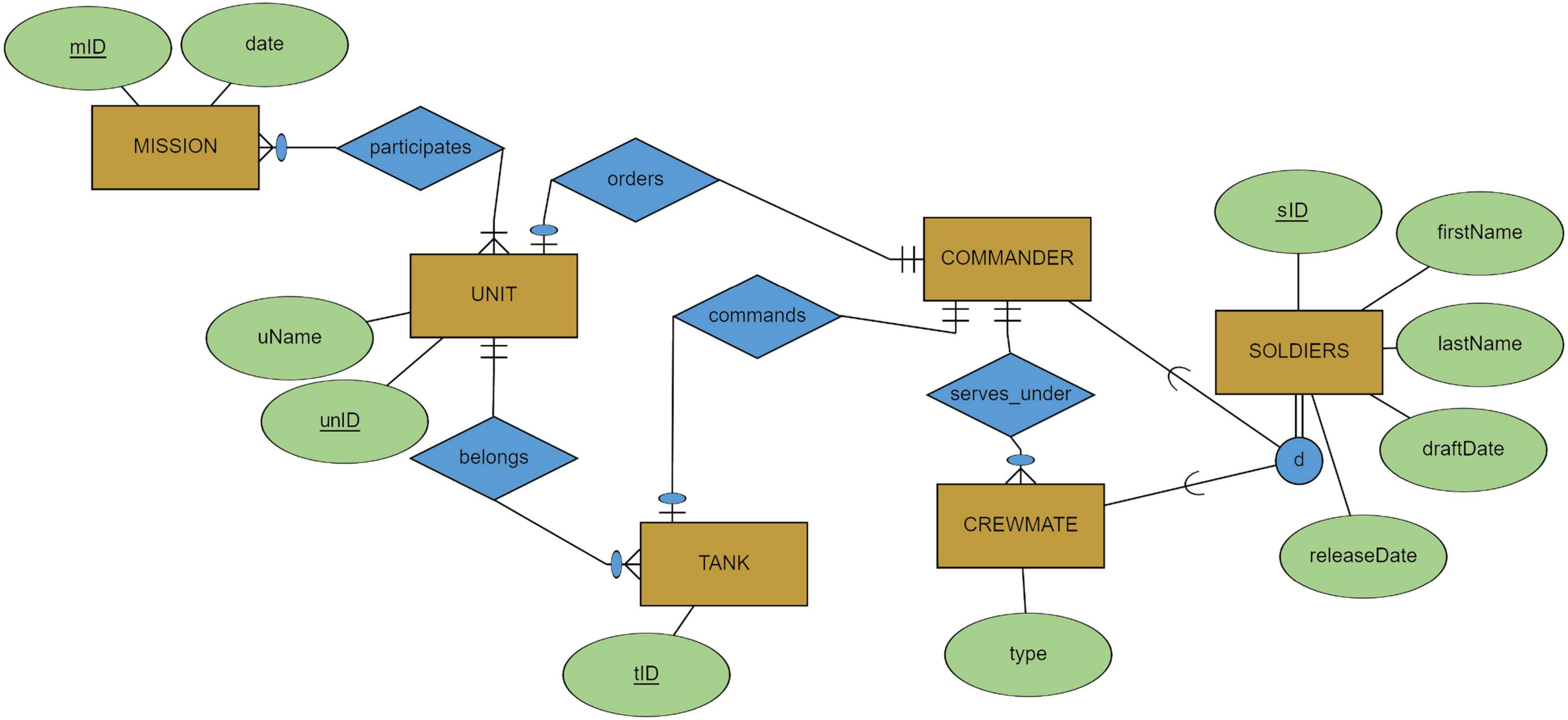
זרוע השריון הישראלית, המוכרת גם כחיל השריון (חש"ן), עומדת כחוד החנית של צבא הגנה לישראל (צה"ל). זרוע זו, מהווה כוח התקפי עיקרי בשדה הקרב, ומשלבת טכנולוגיה מתקדמת, אומץ לב ורוח לחימה עזה.

זרוע השריון היא זרוע עוצמתית וחיונית בצה"ל, הממלאת תפקיד מרכזי בהגנת המדינה. כוח השריון מאפשר לצה"ל לבצע מגוון רחב של משימות בשדה הקרב, ולהוות גורם הרתעה משמעותי מול אויביה.

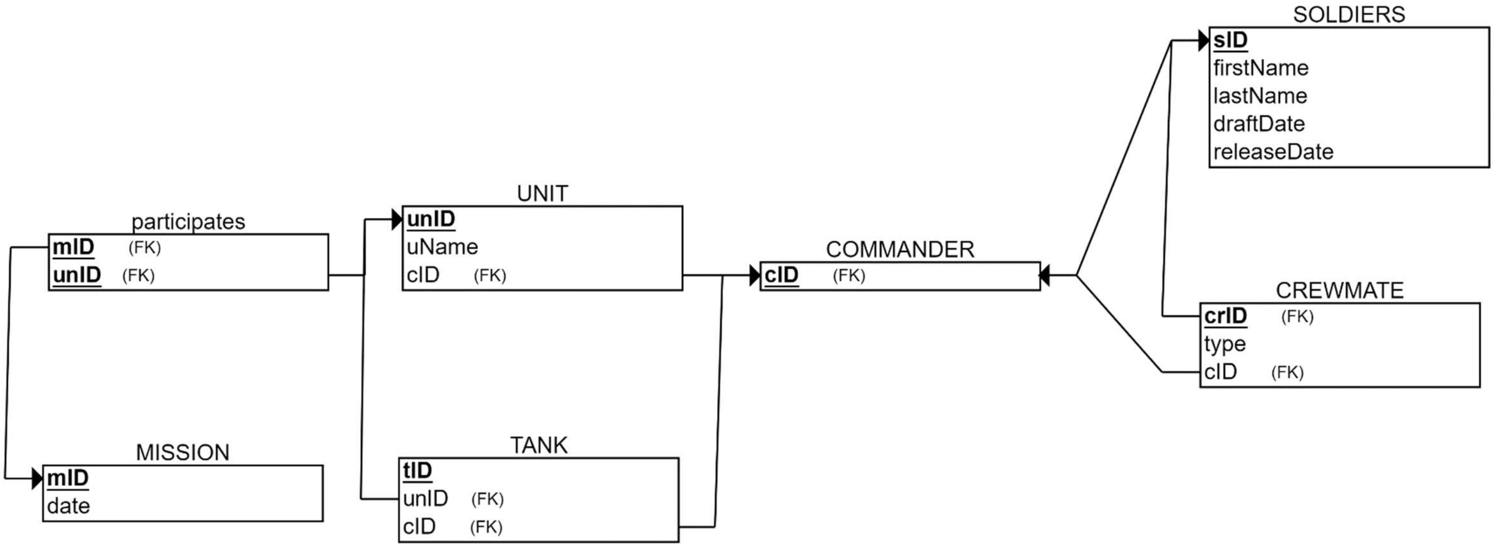
המערכת שנבנה תספק בסיס נתונים לחיל השריון. היא תתחלק לשלושה חלקים:

1. ניהול מידע על החיילים – המערכת תשמור עבור כל חייל את פרטיו האישיים ואת המפקד שלו. עבור מפקד המערכת תשמור גם את תחום פיקודו.
2. ניהול יחידות – המערכת תספק מידע לכל יחידה. שם היחידה, המפקד שלה והטנקים העומדים לרשותה.
3. ניהול משימות – עבור כל משימה המערכת תשמור את זמני המשימה והיחידות המשתתפות

תרשים ERD של הישויות והקשרים



תרשים DSD



תיאור הישויות והתכונות שלהן

SOLIDERS (חיילים)	קבוצה של כל החיילים
sID	תעודת זהות
firstName	שם פרטי
lastName	שם משפחה
draftDate	תאריך גיוס
releaseDate	תאריך שחרור

COMMANDER (מפקד)	מפקד על טנק או על יחידה. יורש מחיילים
cID	מפתח, תעודת זהות

CREWMATE (איש צוות)	איש צוות בתוך טנק. יורש מחיילים
crID	מפתח, תעודת זהות
crID	מפתח זר, תעודת הזהות של מפקד הצוות
type	סוג החייל (נהג, טען או תותחן)

UNIT (יחידה)	יחידה בחיל השריון
unID	מפתח, מספר יחידה מזהה
cID	מפתח זר, תעודת הזהות של מפקד היחידה
uName	שם היחידה

TANK (טנק)	טנק בחיל השריון
<u>tID</u>	מפתח, מספר טנק מזהה
cID	מפתח זר, תעודת הזהות של מפקד הטנק
unID	מפתח זר, מספר היחידה המזהה של הטנק

MISSION (משימה)	משימה בחיל השריון
<u>mID</u>	מפתח, מספר משימה מזהה
mDate	התאריך בו יוצאת המשימה לפועל

תיאור הקשרים

participates (משתתף)	רבים לרבים - יחידה <u>שמשתתפת</u> במשימה
<u>mID</u>	מפתח, מספר המשימה מזהה
<u>unID</u>	מפתח, מספר היחידה מזהה

1. serves_under (משרת) – יחיד לרבים, איש צוות <u>שמשרת</u> תחת מפקד.
2. commands (מפקד) – יחיד לרבים, מפקד <u>שמפקד</u> על טנק.
3. belongs (משתייך) – יחיד לרבים, טנק <u>ששייך</u> ליחידה.
4. orders (נותן הוראות) – יחיד לרבים, מפקד <u>שנותן הוראות</u> ליחידה.

רשימת הסכמות של בסיס הנתונים

הסכמות של הישויות

SOLIDERS(s<u>ID</u>,firstName,lastName,draftDate,releaseDate)
COMMANDER(<u>cID</u>)
CREWMATE(<u>crID</u>,cID,type)
TANK(<u>tID</u>,unID,cID)
UNIT(<u>unID</u>,cID,uName)
MISSION(<u>mID</u>,mdate)

הסכמות של הקשרים

participates(<u>mID,unID</u>)

הוכחה שהסכמות מנורמלות ב3NF

SOLDIERS (חיילים) – המפתח הראשי הוא sID. וכל שאר התכונות תלויות ישירות במפתח הראשי sID. ולכן הטבלה ב3NF.

COMMANDER (מפקד) – cID הוא מפתח ראשי וגם מפתח זר המפנה ל-sID בטבלה SOLDIERS. אין מאפיינים נוספים ולכן הטבלה ב3NF.

CREWMATE (איש צוות) – type תלוי ב-cID וגם crID גם קשור ישירות ל-crID כמפתח זר. הטבלה ב3NF.

UNIT (יחידה) – uName ו-cID תלויים במפתח הראשי unID. הטבלה נמצאת ב3NF.

TANK (טנק) – unID ו-cID הם תכונות עם יחסים של מפתחות זרים. אבל, אין תלות טרנזיטיבית כיוון ש-tID הוא המפתח הראשי. ולכן הטבלה ב3NF.

MISSION (משימה) – מכיוון שmdatew תלוי ישירות במפתח הראשי mID, הטבלה ב3NF.

participates(משתתף) – גם mID וגם unID הם מפתחות זרים, והם יוצרים יחד מפתח מורכב. כל מאפיין במפתח המורכב קובע לחלוטין את המאפיינים האחרים.

למסקנה כל הטבלאות עונות על התנאים ל-3NF.

קוד SQL של יצירת הטבלאות (CREATE)

```sql
CREATE TABLE SOLDIERS
(
  sID numeric(9) NOT NULL,
  firstName VARCHAR(20) NOT NULL,
  lastName VARCHAR(20) NOT NULL,
  draftDate DATE NOT NULL,
  releaseDate DATE NOT NULL,
  PRIMARY KEY (sID)
);

CREATE TABLE MISSION
(
  mdate DATE NOT NULL,
  mID numeric(9) NOT NULL,
  PRIMARY KEY (mID)
);

CREATE TABLE COMMANDER
(
  cID numeric(9) NOT NULL,
```

```sql
  PRIMARY KEY (cID),
  FOREIGN KEY (cID) REFERENCES SOLDIERS(sID)
);

CREATE TABLE CREWMATE
(
  type VARCHAR(20) NOT NULL,
  crID numeric(9) NOT NULL,
  cID numeric(9) NOT NULL,
  PRIMARY KEY (crID),
  FOREIGN KEY (crID) REFERENCES SOLDIERS(sID),
  FOREIGN KEY (cID) REFERENCES COMMANDER(cID)
);

CREATE TABLE UNIT
(
  unID numeric(9) NOT NULL,
  uName VARCHAR(20) NOT NULL,
  cID numeric(9) NOT NULL,
  PRIMARY KEY (unID),
  FOREIGN KEY (cID) REFERENCES COMMANDER(cID)
);

CREATE TABLE TANK
(
  tID numeric(9) NOT NULL,
  unID numeric(9) NOT NULL,
  cID numeric(9) NOT NULL,
  PRIMARY KEY (tID),
  FOREIGN KEY (unID) REFERENCES UNIT(unID),
  FOREIGN KEY (cID) REFERENCES COMMANDER(cID)
);

CREATE TABLE participates
(
  mID numeric(9) NOT NULL,
  unID numeric(9) NOT NULL,
  PRIMARY KEY (mID, unID),
  FOREIGN KEY (mID) REFERENCES MISSION(mID),
  FOREIGN KEY (unID) REFERENCES UNIT(unID)
);
```

קוד SQL של זריקת הטבלאות (DROP)

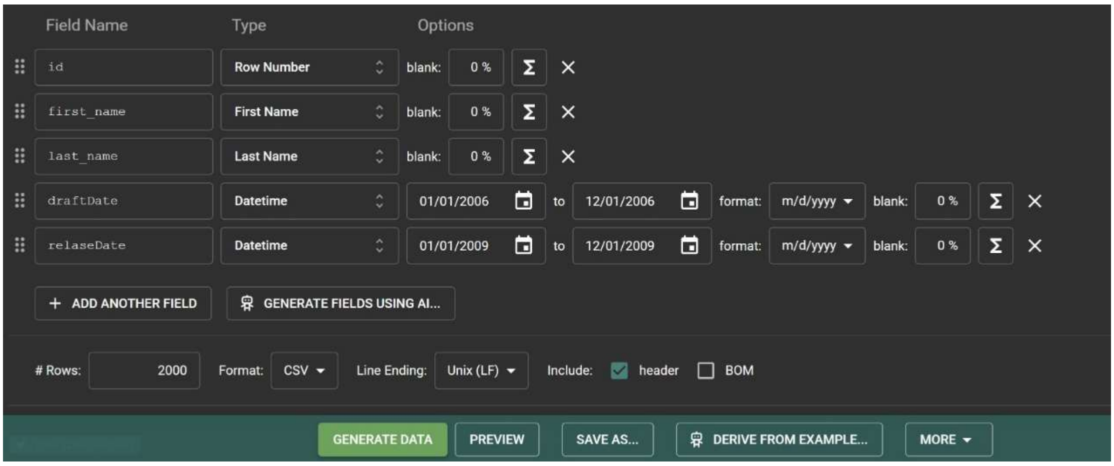
```sql
DROP TABLE CREWMATE;
DROP TABLE TANK;
DROP TABLE participates;
DROP TABLE MISSION;
DROP TABLE UNIT;
DROP TABLE COMMANDER;
DROP TABLE SOLDIERS;
```

הכנסת מידע לטבלאות

נכניס את כל הנתונים שלנו לתוך מסד הנתונים. לשם כך נוכל להשתמש בשיטות שונות.

הכנסת מידע לSOLIDERS – דרך 1, Mockaroo file

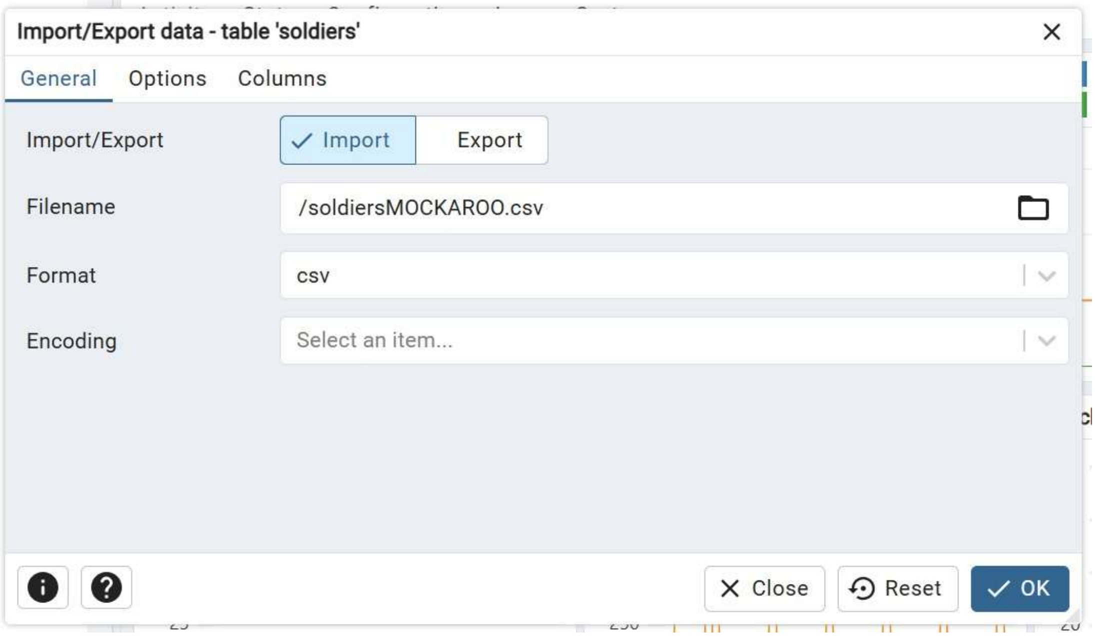
ניצור ונכניס את הData בעזרת מחולל המידע של Mockaroo file



ככה נראה המידע המפוברק שקיבלנו ממחולל הנתונים.

F	E	D	C	B	A
	relaseDate	draftDate	last_name	first_name	id
	10/31/2009	5/13/2006	Shuttlewoc	Sharlene	1
	########	########	Clue	Olwen	2
	2/22/2009	########	Worthingtc	Chris	3
	2/25/2009	2/18/2006	Monketon	Niccolo	4
	9/30/2009	1/31/2006	Lappine	Shelba	5
	########	11/30/2006	Hendrik	Lina	6
	2/19/2009	########	MacAdam	Erina	7
	########	########	Showell	Sapphira	8
	1/27/2009	5/13/2006	Bonome	Clair	9
	8/14/2009	6/20/2006	Decourcy	Sophie	10
	10/28/2009	########	Gilliatt	Clarissa	11
	6/16/2009	10/17/2006	Farrey	Rowan	12
	3/16/2009	########	Virgo	Grier	13

נייבא את קובץ ה-csv בעזרת מערכת הקבצים של Postgres



נבדוק שהמידע אכן נקלט במערכת בעזרת הפקודה Select * from SOLDIERS

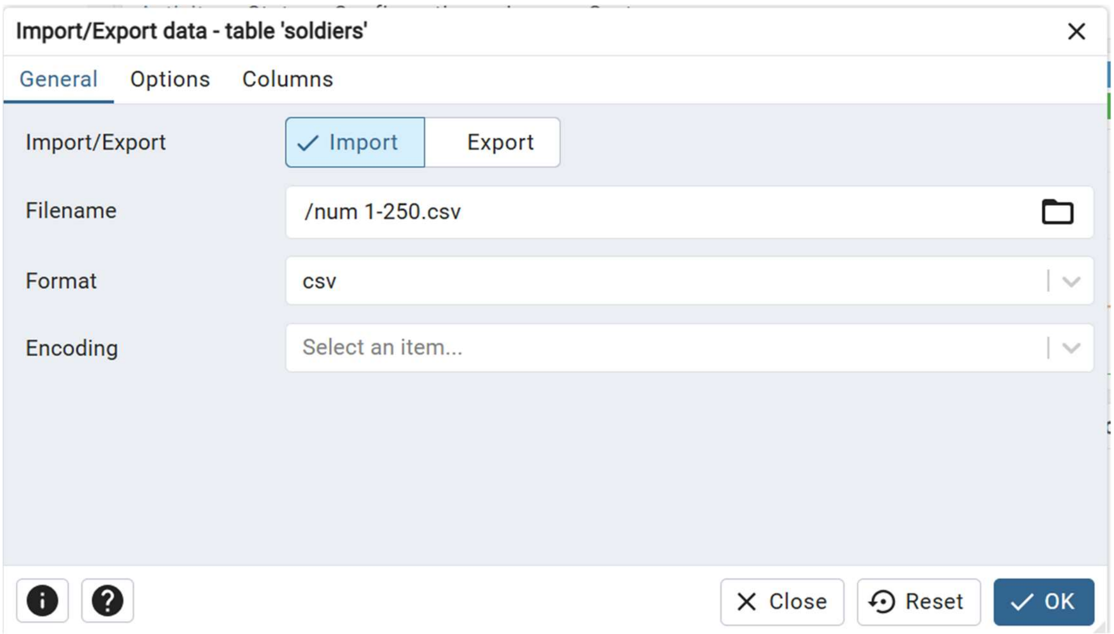
91	91	Jesse	Mewes	2006-09-26	2009-04-15
92	92	Kay	Sizemore	2006-04-13	2009-11-14
93	93	Lili	Sirtis	2006-09-28	2009-07-05
94	94	Dylan	Curtis	2006-02-15	2009-10-22
95	95	Horace	McConaughey	2006-02-12	2009-08-23
96	96	Dorry	Pleasure	2006-01-05	2009-12-12
97	97	Kris	Tilly	2006-05-13	2009-02-22
98	98	Harris	Visnjic	2006-11-22	2009-11-25
99	99	Courtney	Reubens	2006-06-27	2009-04-10
100	100	Chely	McFadden	2006-04-13	2009-05-07
101	101	Kazem	Hewett	2006-03-06	2009-01-03
102	102	Derek	Palminteri	2006-04-24	2009-06-13
103	103	Mac	Weiss	2006-04-13	2009-06-25
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

הכנסת מידע לCOMMANDER – דרך 2, קובץ Excel

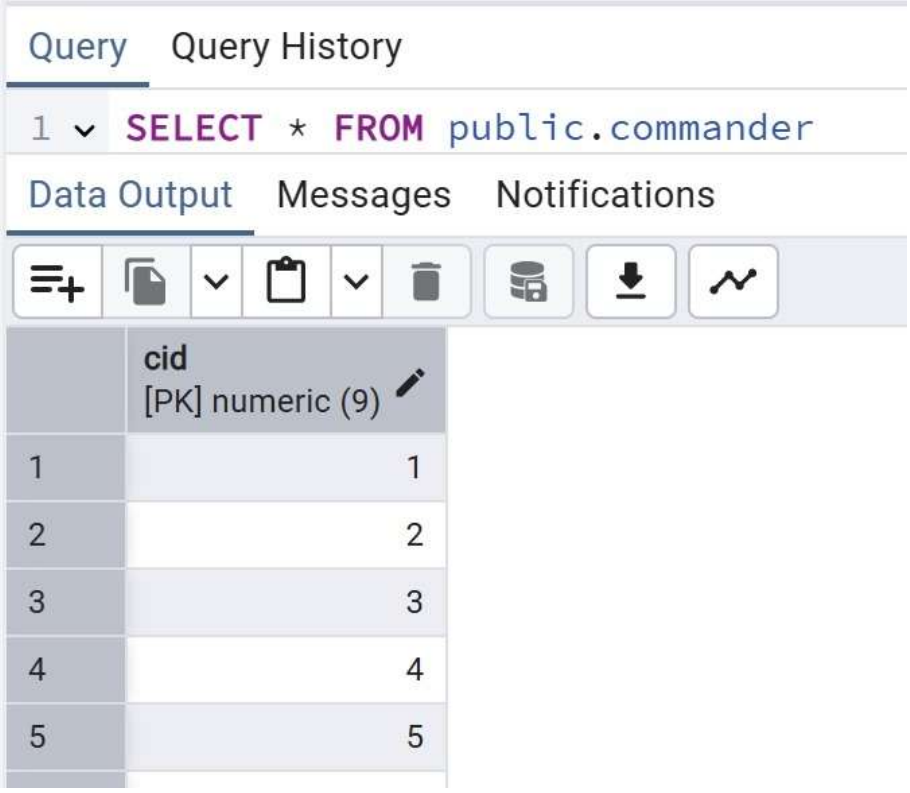
נשתמש במידע מתוך קובץ Excel

	A	B
1	id	
2	1	
3	2	
4	3	
5	4	
6	5	
7	6	
8	7	
9	8	
10	9	
11	10	
12	11	
13	12	
14	13	
15	14	

נייבא את קובץ ה-csv בעזרת מערכת הקבצים של Postgres



נבדוק האם המידע נקלט בעזרת הפקודה Select * from COMMANDER



הכנסת מידע לCREWMATE – דרך 3, סקריפט בפייתון

הפעם ניקח דרך שונה. ניצור סקריפט בפייתון שיצור רשימה של פקודות INSERT בהן נשתמש בשביל להכניס מידע לאנשי הצוות.

```python
types = ['Loader','Gunner','Driver']

def generate_crewmate_data(num_records):
    data = []
    for x in range(num_records):
        id = x + 100
        cid = int((id - 100) / 3 ) + 1; # getting the id of the commander
        type = types[x % 3 ]
        data.append((id,cid, type))
    return data

def generate_sql_insert_statements(table_name, data):
    sql_statements = []
    for record in data:
        sql = f"INSERT INTO {table_name} (TYPE, CRID, CID) VALUES ({record[2]},{record[0]},{record[1]});"
        sql_statements.append(sql)
    return sql_statements

# Generate data
num_records = 1400  # Number of records we want to generate
crewmate_data = generate_crewmate_data(num_records)

# Generate SQL insert statements
sql_statements = generate_sql_insert_statements( table_name: 'CREWMATE', crewmate_data)
```

נכניס את שורות הInsert אל לתוך מסך הנתונים

```sql
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Loader', 100,1);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Gunner', 101,1);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Driver', 102,1);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Loader', 103,2);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Gunner', 104,2);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Driver', 105,2);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Loader', 106,3);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Gunner', 107,3);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Driver', 108,3);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Loader', 109,4);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Gunner', 110,4);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Driver', 111,4);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Loader', 112,5);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Gunner', 113,5);
INSERT INTO CREWMATE (TYPE, CRID, CID) VALUES ('Driver', 114,5);
```

tput Messages Notifications

ונוודא שמידע נקלט בעזרת פקודת SELECT

Query | Query History

```sql
SELECT * FROM public.crewmate
ORDER BY crid ASC
```

Data Output | Messages | Notifications

	type character varying (20)	crid [PK] numeric (9)	cid numeric (9)
1	Loader	601	1
2	Gunner	602	1
3	Driver	603	1
4	Loader	604	2
5	Gunner	605	2

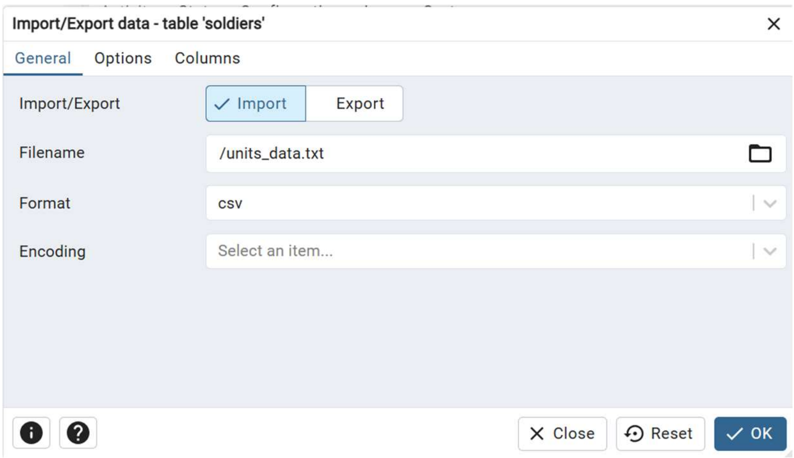
הכנסת מידע לUNIT – דרך 4, קובץ טקסט

הפעם נכניס למסד הנתונים בטבלה UNIT מידע מקובץ טקסט, ניעזר בבינה מלאכותית ובסיס הנתונים של צה"ל על מנת להכניס מידע ריאליסטי ככל הניתן.

File Edit Format View Help

```
1, 351, Saar me-Golan
2, 352, Barak
3, 353, Iron tails
4, 354, Sons of light
5, 355, Utzvat HaPlada
6, 356, Ikvot HaBarzel
7, 357, Bnei Or
8, 358, Kiryati
9, 359, Machatz
10, 360, Ram
11, 361, Iron Fist
12, 362, Yiftach
13, 363, Sinai
14, 364, Harel
```

נייבא את קובץ ה-csv בעזרת מערכת הקבצים של Postgres



נוודא שאכן המידע נשמר במערכת

```sql
SELECT * FROM public.unit
ORDER BY unid ASC
```

Data Output Messages Notifications

	unid [PK] numeric (9)	uname character varying (20)	cid numeric (9)
1	1	Saar me-Golan	351
2	2	Barak	352
3	3	Iron tails	353
4	4	Sons of light	354
5	5	Utzvat HaPlada	355
6	6	Ikvot HaBarzel	356
7	7	Bnei Or	357
8	8	Kiryati	358
9	9	Machatz	359

הכנסת מידע לTANK – דרך 3, סקריפט בפייתון

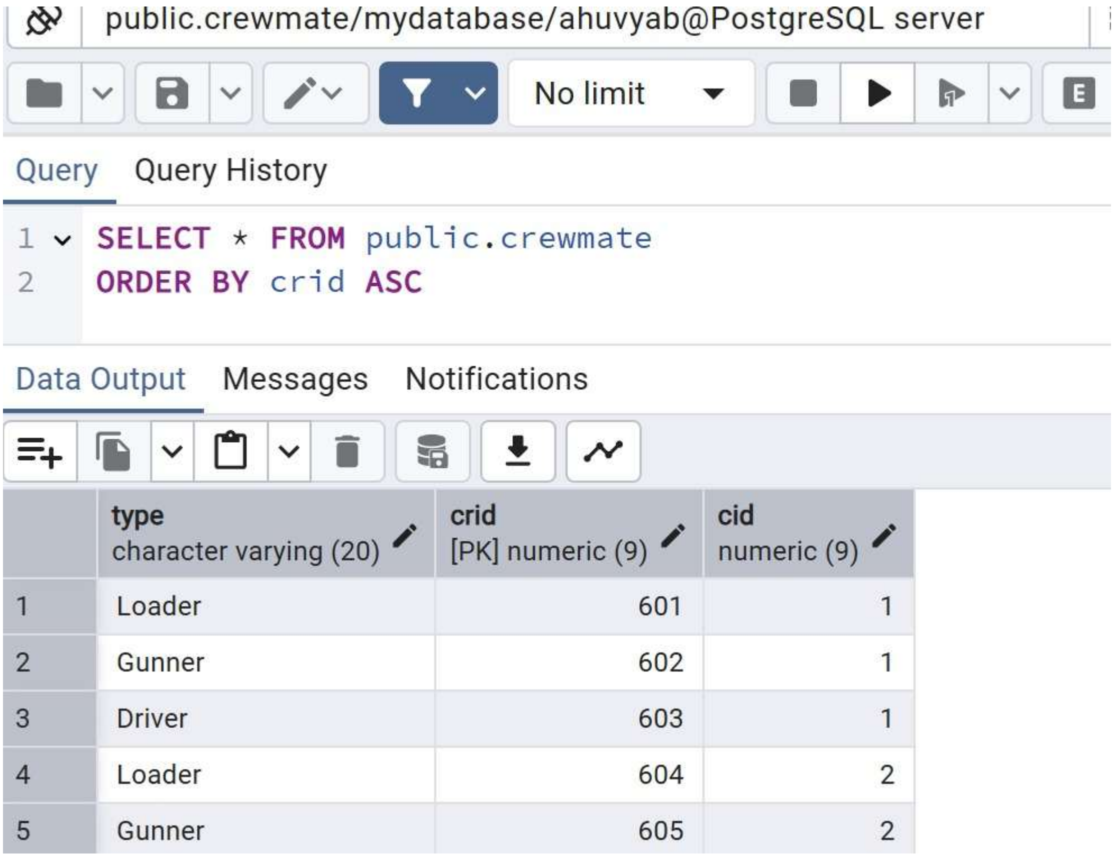
```python
import random

def generate_tank_data(num_records):
    data = []
    for x in range(num_records):
        id = x + 1
        cid = random.randrange(1, 250)
        unid = random.randrange(1, 250) # getting the id of the unit
        data.append((id,cid, unid))
    return data

def generate_sql_insert_statements(table_name, data):
    sql_statements = []
    for record in data:
        sql = f"INSERT INTO {table_name} (TID, UNID, CID) VALUES ({record[0]},{record[2]},{record[1]});"
        sql_statements.append(sql)
    return sql_statements

# Generate data
num_records = 350  # Number of records we want to generate
tank_data = generate_tank_data(num_records)

# Generate SQL insert statements
sql_statements = generate_sql_insert_statements('TANK', tank_data)
```

הסקריפט יוצר לנו פקודות INSERT בSQL

נריץ את הפקודות דרך ה query tool ב Postgres

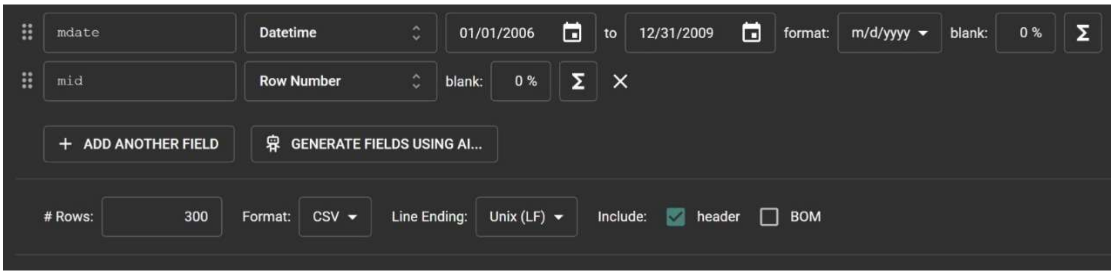
```sql
INSERT INTO TANK (TID, UNID, CID) VALUES (1,506,75);
INSERT INTO TANK (TID, UNID, CID) VALUES (2,478,55);
INSERT INTO TANK (TID, UNID, CID) VALUES (3,529,141);
INSERT INTO TANK (TID, UNID, CID) VALUES (4,586,115);
INSERT INTO TANK (TID, UNID, CID) VALUES (5,586,144);
INSERT INTO TANK (TID, UNID, CID) VALUES (6,457,63);
INSERT INTO TANK (TID, UNID, CID) VALUES (7,402,130);
INSERT INTO TANK (TID, UNID, CID) VALUES (8,455,122);
INSERT INTO TANK (TID, UNID, CID) VALUES (9,549,21);
INSERT INTO TANK (TID, UNID, CID) VALUES (10,429,82);
INSERT INTO TANK (TID, UNID, CID) VALUES (11,439,37);
INSERT INTO TANK (TID, UNID, CID) VALUES (12,549,9);
INSERT INTO TANK (TID, UNID, CID) VALUES (13,479,17);
INSERT INTO TANK (TID, UNID, CID) VALUES (14,586,10);
```

נוודא שהמידע נקלט במערכת בעזרת שאילתת ;select * from TANK

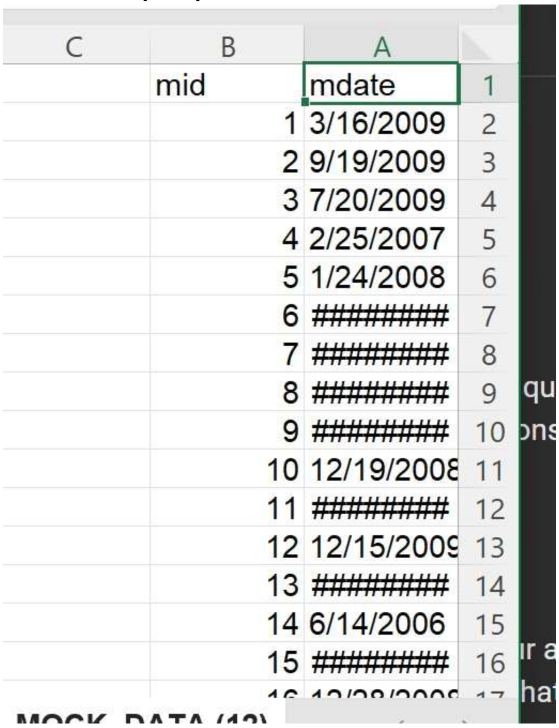


הכנסת מידע לMISSION – דרך 1, Mockaroo file

ניצור ונכניס את הData בעזרת מחולל המידע של Mockaroo file

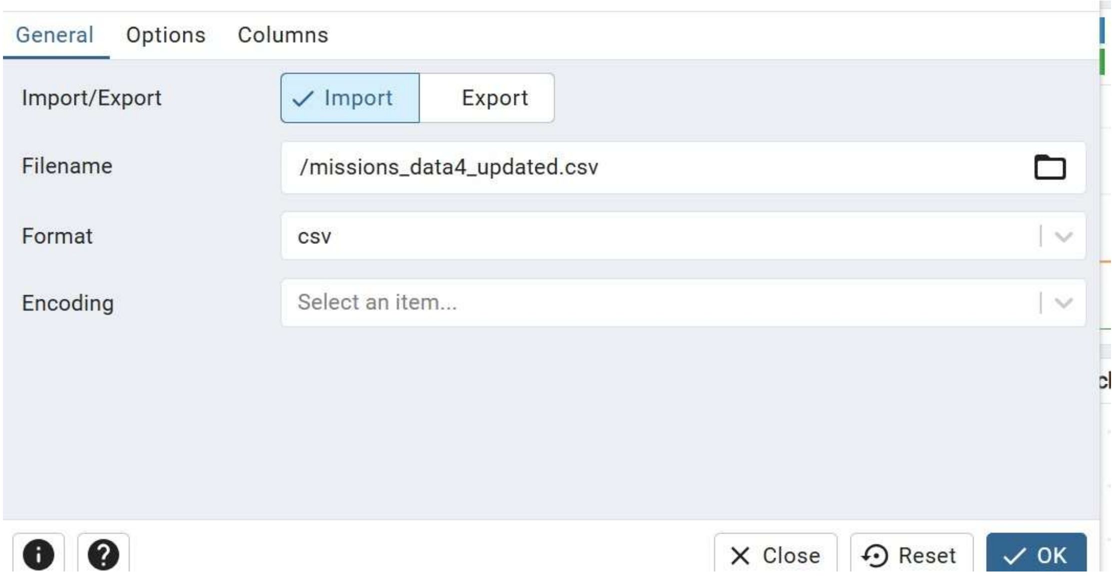


ככה נראה המידע המפוברק שקיבלנו ממחולל הנתונים.

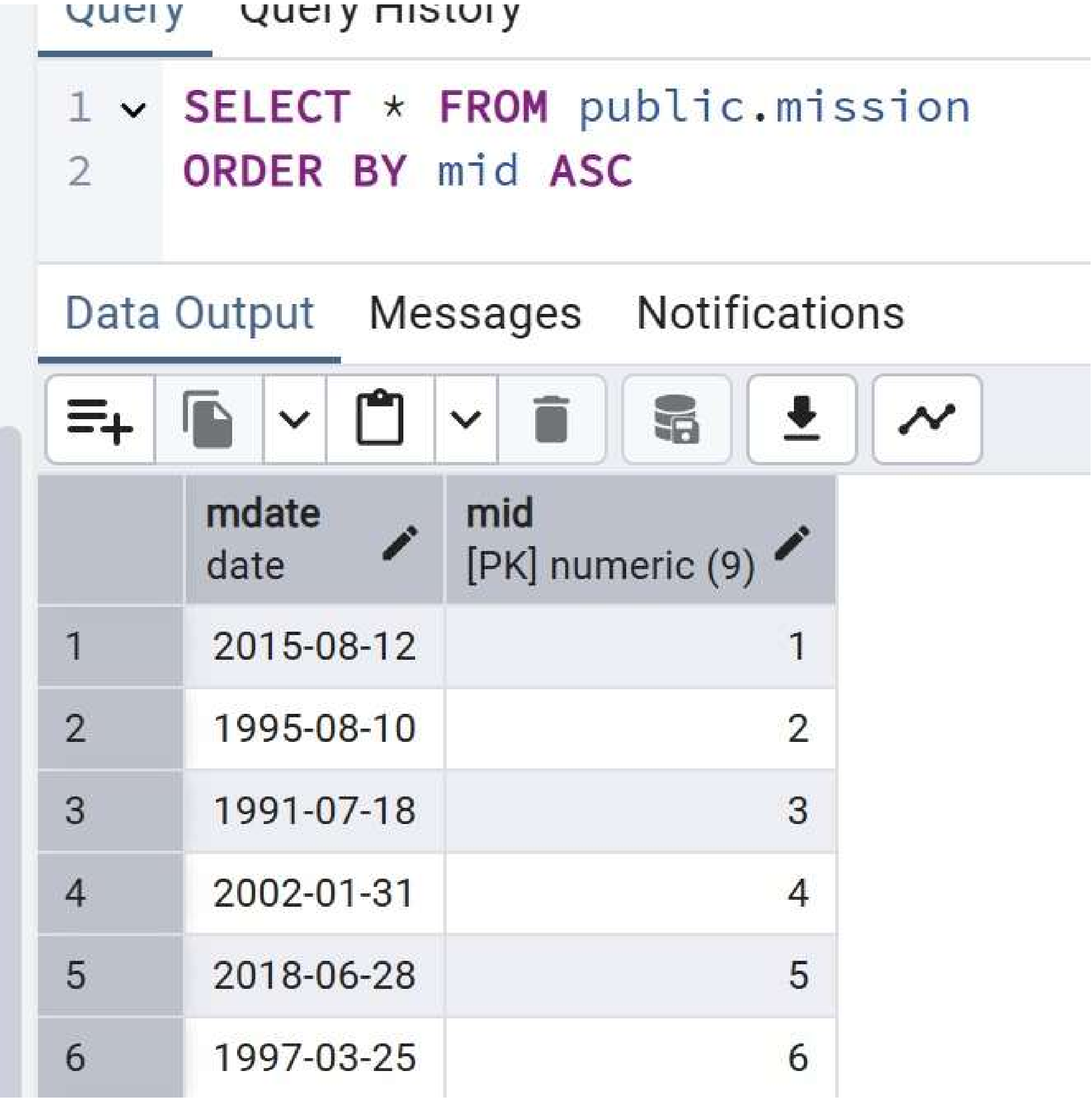


	A	B	C
1	mdate	mid	
2	3/16/2009	1	
3	9/19/2009	2	
4	7/20/2009	3	
5	2/25/2007	4	
6	1/24/2008	5	
7	########	6	
8	########	7	
9	########	8	
10	########	9	
11	12/19/2008	10	
12	########	11	
13	12/15/2009	12	
14	########	13	
15	6/14/2006	14	
16	########	15	

נייבא את קובץ ה-csv בעזרת מערכת הקבצים של Postgres



נבדוק שהמידע אכן נקלט במערכת בעזרת הפקודה Select * from mission

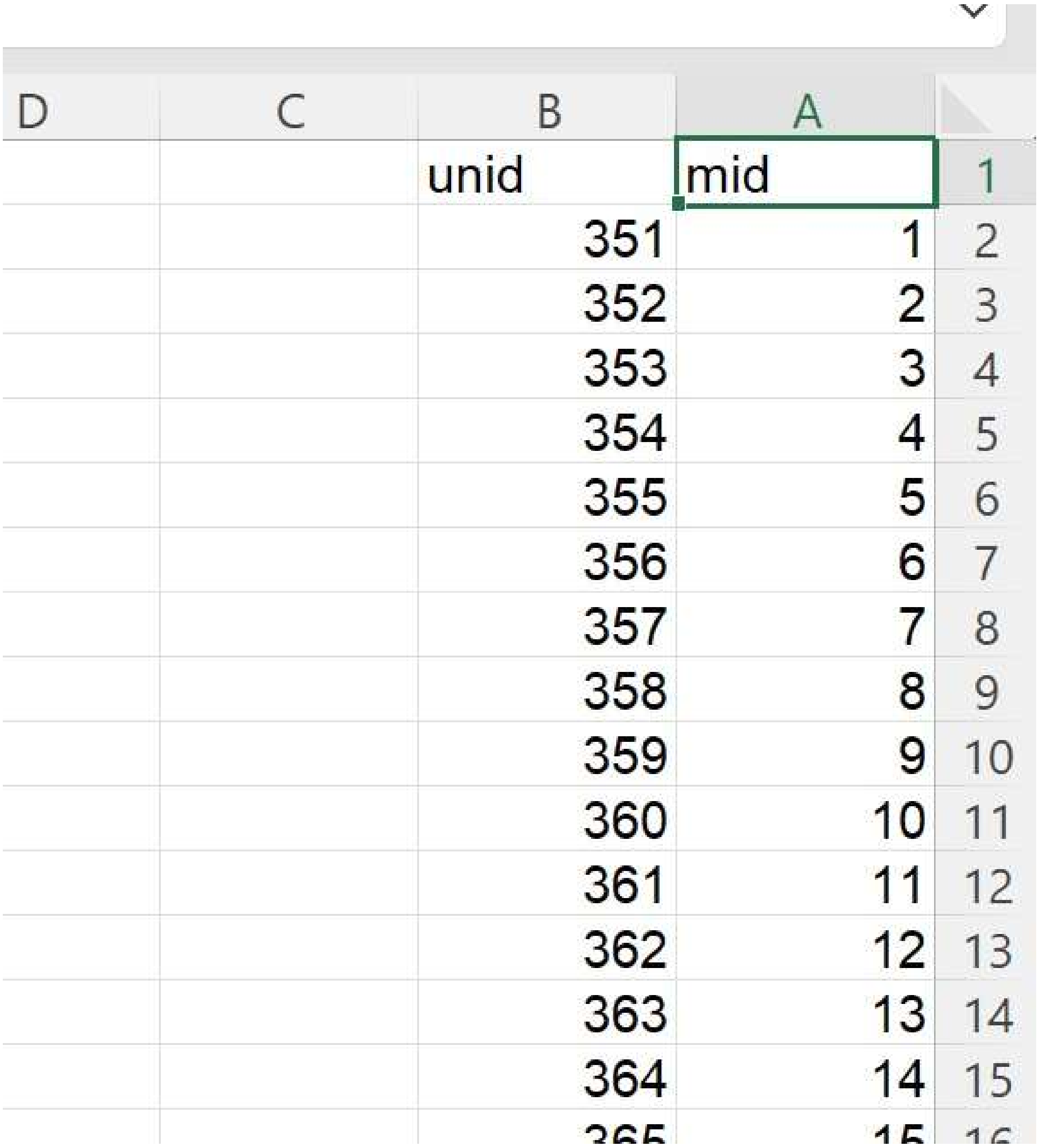



```sql
SELECT * FROM public.mission
ORDER BY mid ASC
```

	mdate date	mid [PK] numeric (9)
1	2015-08-12	1
2	1995-08-10	2
3	1991-07-18	3
4	2002-01-31	4
5	2018-06-28	5
6	1997-03-25	6

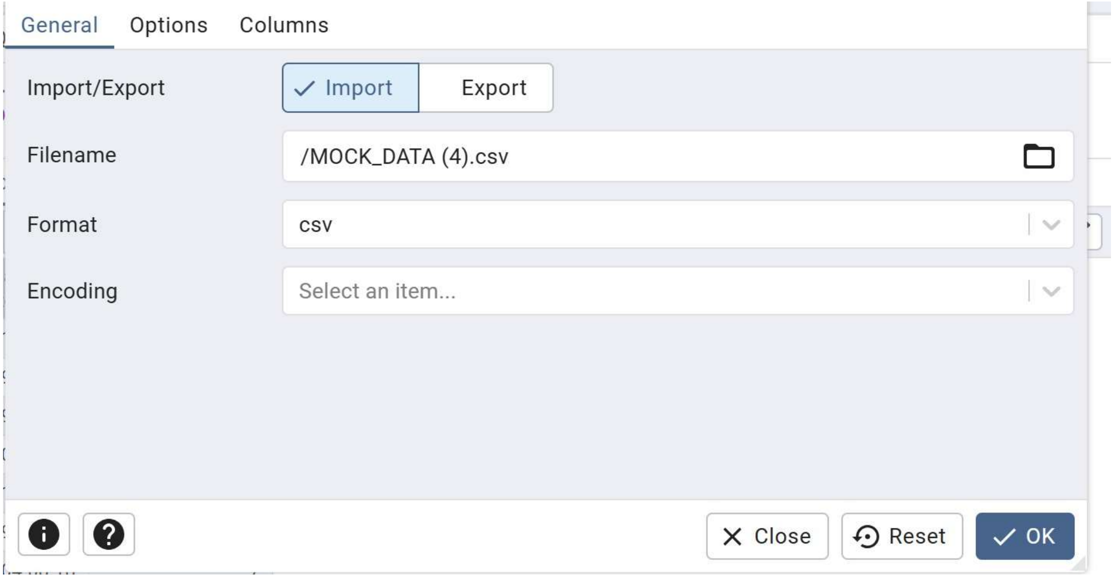
הכנסת מידע לparticipate – דרך 2, קובץ Excel

נשתמש במידע מתוך קובץ Excel

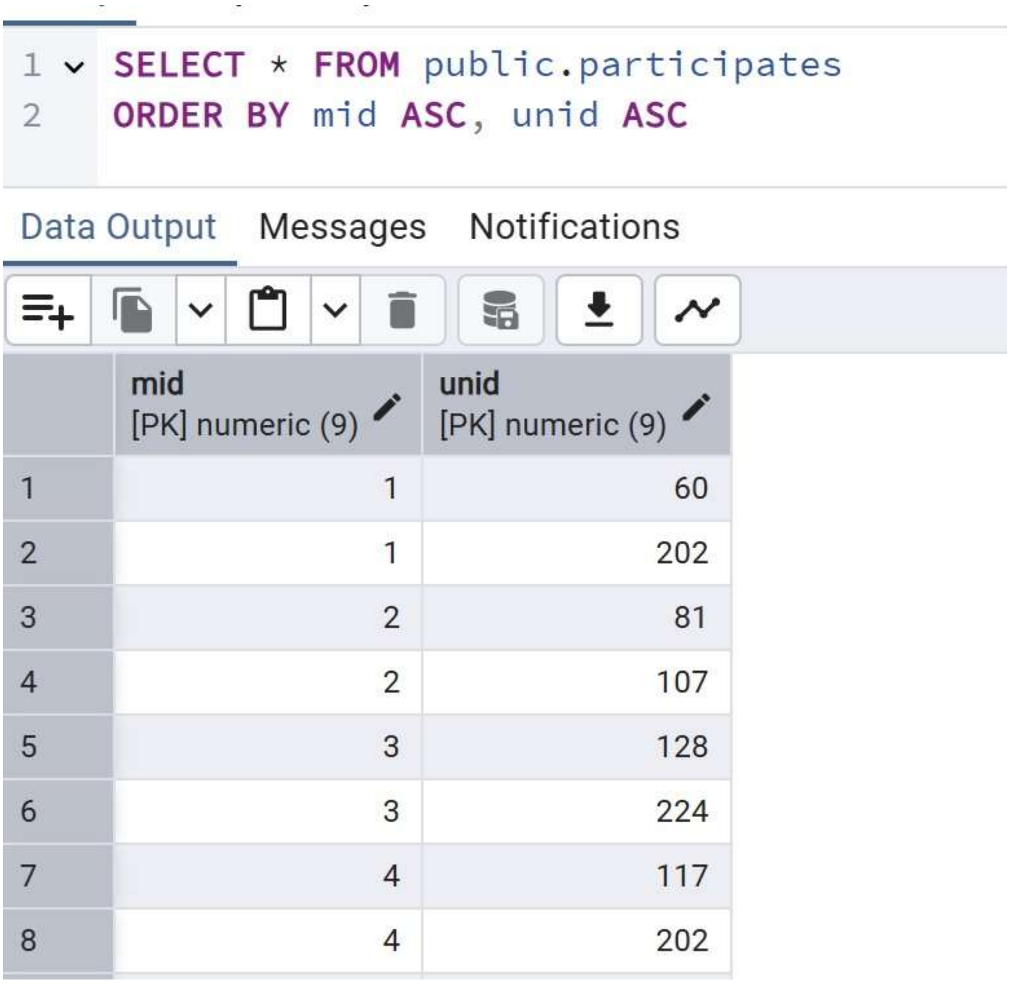


	A	B	C	D
1	mid	unid		
2	1	351		
3	2	352		
4	3	353		
5	4	354		
6	5	355		
7	6	356		
8	7	357		
9	8	358		
10	9	359		
11	10	360		
12	11	361		
13	12	362		
14	13	363		
15	14	364		
16	15	365		

נייבא את קובץ ה-csv בעזרת מערכת הקבצים של Postgres



נבדוק האם המידע נקלט בעזרת הפקודה Select * from participates



```
SELECT * FROM public.participates
ORDER BY mid ASC, unid ASC
```

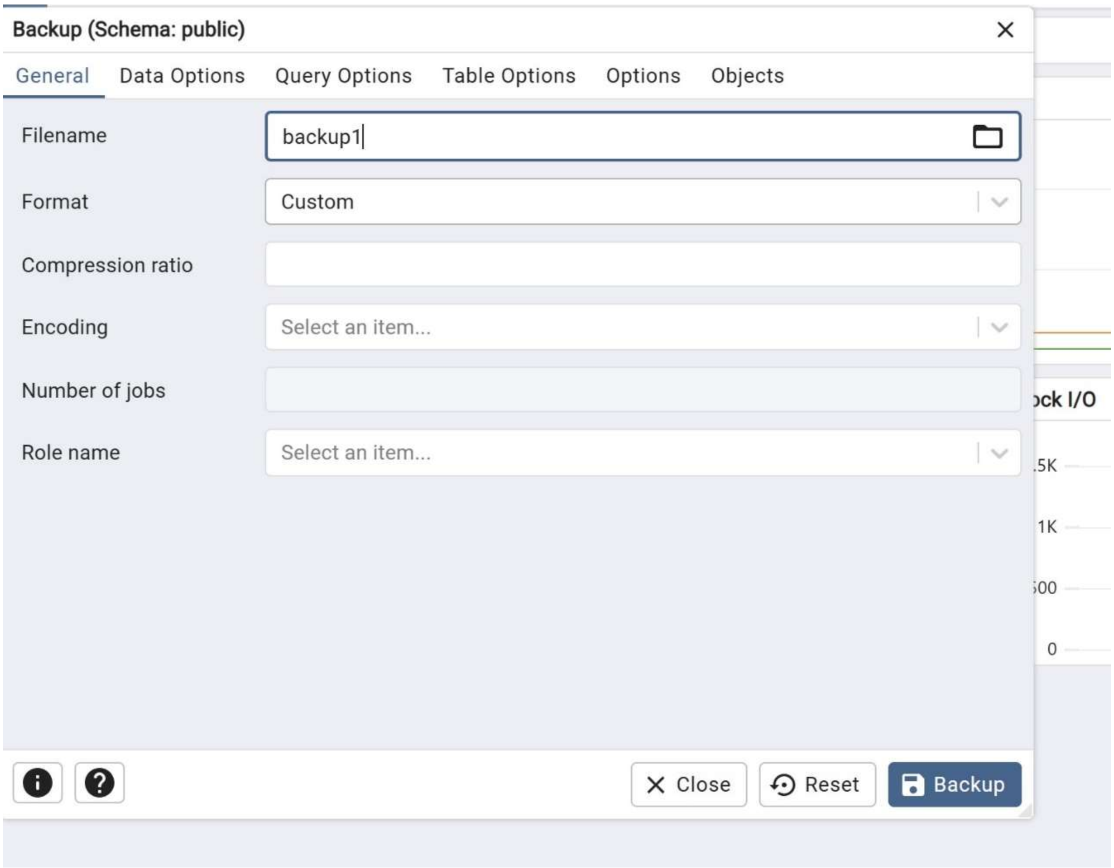
Data Output Messages Notifications

	mid [PK] numeric (9)	unid [PK] numeric (9)
1	1	60
2	1	202
3	2	81
4	2	107
5	3	128
6	3	224
7	4	117
8	4	202

גיבוי הטבלאות

בשביל להבטיח שהמידע של בסיס הנתונים שלנו תמיד נשמר נצטרך לגבות את כל המידע שבתוך הטבלאות למקור שמירה חיצוני.

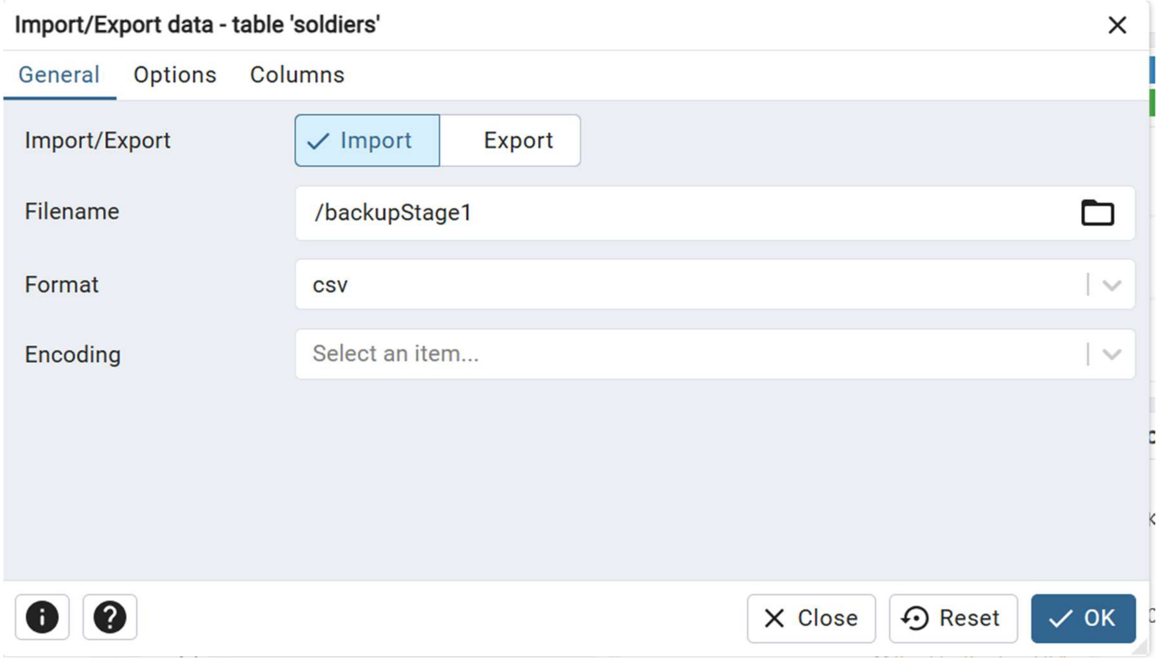
אחרי שסימנו את כל הטבלאות שאנחנו רוצים לשמור, נייצא את המידע שבהן לתוך קובץ מסוג יצוא וככה נוכל גם להעביר את הטבלאות למחשבים אחרים וגם לשמור עליהן



שחזור הטבלאות

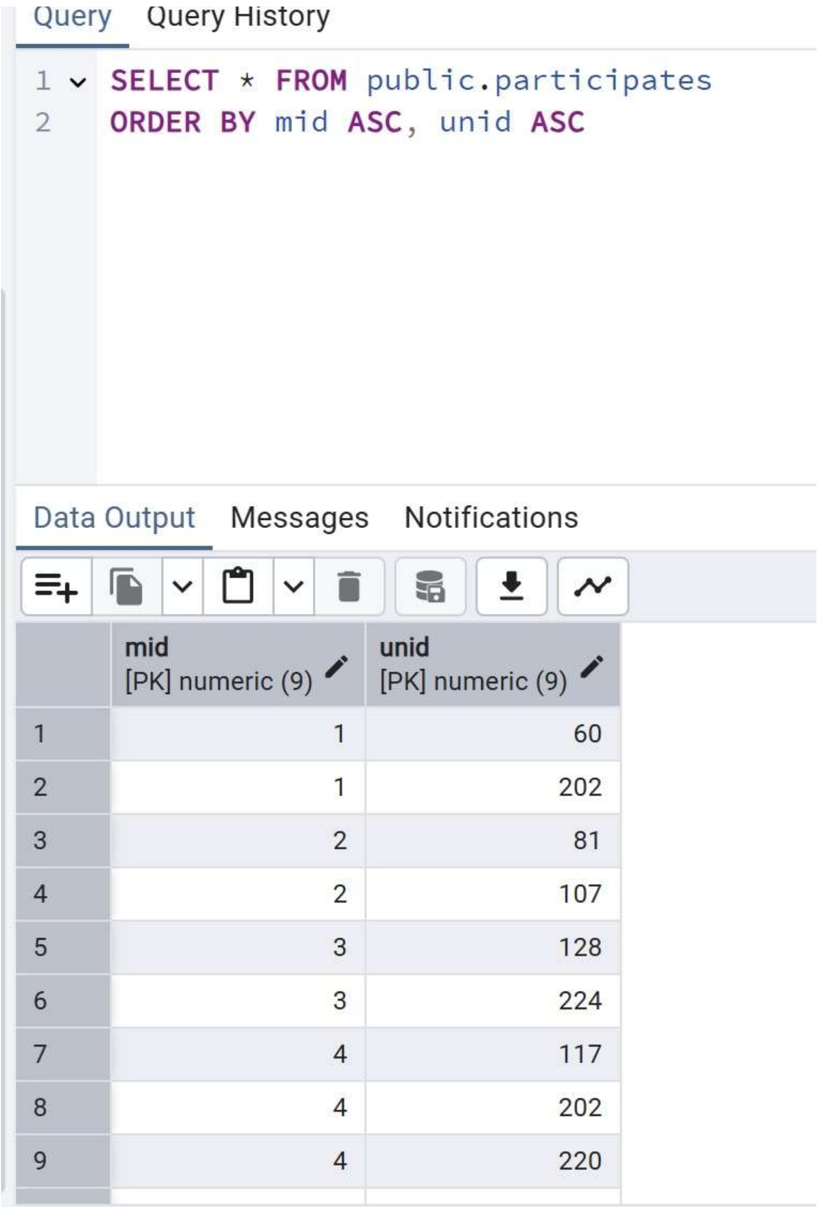
אחרי שגיבינו את כל הטבלאות, נרצה לראות שזה אכן עבד. החלטנו למחוק את המידע שבטבלה participates בעזרת הפקודה Delete.

נטען עכשיו את המידע מקובץ היצוא החיצוני שלנו בשביל לבדוק האם המידע באמת נשמר



המידע נשמר 🥳

אנחנו יכולים לגשת אליו גם אחרי שמחקנו אותו



Query | Query History

```sql
SELECT * FROM public.participates
ORDER BY mid ASC, unid ASC
```

Data Output | Messages | Notifications

	mid [PK] numeric (9)	unid [PK] numeric (9)
1	1	60
2	1	202
3	2	81
4	2	107
5	3	128
6	3	224
7	4	117
8	4	202
9	4	220

שלב 2 - שאילתות (וכן השאילתות עם פרמטרים)

שאילתות

8 שאילתות select:

שאילתא 1 : מציאת זמן השירות הממוצע של חיילים ששירתו בתור מפקדים, והשוואה עם זמן השירות הממוצע של אילו שאינם מפקדים.

```
-- Subquery for Commanders
SELECT 'Commander' AS role,
    AVG(s.releaseDate - s.draftDate) AS
avgServiceDuration
FROM SOLDIERS s
JOIN COMMANDER c ON s.sID = c.cID

UNION ALL

-- Subquery for Non-Commanders
SELECT 'Non-Commander' AS role,
    AVG(s.releaseDate - s.draftDate) AS
avgServiceDuration
FROM SOLDIERS s
LEFT JOIN COMMANDER c ON s.sID = c.cID
WHERE c.cID IS NULL;
```

התוצאה שתתקבל מהרצת השאילתה היא

	role text	avgserviceduration numeric
1	Commander	1097.9016666666666667
2	Non-Commander	1090.5742857142857143

Total rows: 2 Query complete 00:00:00.069

שאילתה 2 : רשימה של כל היחידות שהשתתפו ביותר משימות ממספר המשימות ליחידה הממוצע

```
SELECT u.unID, u.uName, COUNT(p.mID) AS missionCount
FROM UNIT u
JOIN participates p ON u.unID = p.unID
GROUP BY u.unID, u.uName
HAVING COUNT(p.mID) > (
    SELECT AVG(missionCount)
    FROM (
        SELECT COUNT(p.mID) AS missionCount
        FROM participates p
        GROUP BY p.unID

    ) subquery
)
ORDER BY missionCount DESC;
```

התוצאה שתתקבל מהרצת השאילתה היא

	unid [PK] numeric (9)	uname character varying (20)	missioncount bigint
1	129	Iron tails 4	8
2	135	Machatz 4	7
3	54	Yiftach 2th	7
4	232	Rays of fire 6	7
5	49	Bnei Or 2th	7
6	20	Galloping Horse	7
7	11	Iron Fist	6
8	158	Netivei Esh 4	6
9	218	Kiryati 6	6
10	200	Netivei Esh 5	6
11	173	Utzvat HaPlada 5	6
12	241	Stars of fire 6	6
13	115	Stars of fire 3	6
14	141	Kvir 4	6
15	44	Barak 2th	6
16	125	Etgar 3	6

Total rows: 129 Query complete 00:00:00.083

שאילתה 3 : כל היחידות ממוינות לפי מספר הטנקים בכל יחידה

```
SELECT u.uName AS unitName, COUNT(t.tID) AS numberOfTanks
FROM UNIT u
LEFT JOIN TANK t ON u.unID = t.unID
GROUP BY u.uName
ORDER BY numberOfTanks DESC;
```

התוצאה שתתקבל מהרצת השאילתה היא

	unitname character varying (20)	numberoftanks bigint
1	Etzion Gever 5	6
2	Yerushalaim 5	6
3	Iron Fist 2th	5
4	Mapatz 3	5
5	Kela 5	5
6	Sons of light 2th	5
7	Eshet 4	4
8	Rays of fire	4
9	Fire's Chariots 3	4
10	Ikvot HaBarzel 3	4
11	Yishai 4	4
12	Hamud HaEsh 5	4
13	Kvir 5	3
14	Arrow 4	3
15	Bnei Reshef 2	3
16	Bazka 4	3

Total rows: 250 Query complete 00:00:00.073

שאילתה 4 : אחרי מבצע מוצלח רצו בפיקוד להביא לחיילים צ'ופר, לכל חייל תיק עם הדפס של היחידה. בשביל כל החיילים שהתגייסו במרץ, הודפס התיק "גיוס מרץ שריון".
יחידת האפסנאות צריכה רשימה של כל החיילים שהתגייסו בסביבות מרץ אפריל.

```
SELECT * FROM CREWMATE
WHERE crID IN (
  SELECT sID
```

```
  FROM SOLDIERS
  WHERE EXTRACT(MONTH FROM draftDate) BETWEEN 3 AND 4
);
```

התוצאה שתתקבל מהרצת השאילתה היא

	type character varying (20)	crid [PK] numeric (9)	cid numeric (9)
1	Loader	1708	370
2	Gunner	1727	376
3	Gunner	1730	377
4	Loader	1732	378
5	Driver	1740	380
6	Loader	1741	381
7	Gunner	1742	381
8	Loader	1744	382
9	Gunner	1745	382
10	Loader	1750	384
11	Gunner	1757	386
12	Driver	1761	387
13	Gunner	1772	391
14	Driver	1785	395
15	Loader	1795	399

Total rows: 217 Query complete 00:00:00.080

שאילתה 5 : מציאת 10 החיילים הוותיקים ביותר.

```
SELECT
  s.firstName || ' ' || s.lastName AS commanderName,
  s.draftDate,
  s.releaseDate,
  FLOOR((s.releaseDate - s.draftDate)/365) AS yearsServed,
  FLOOR(MOD((s.releaseDate - s.draftDate), 365)/30) AS monthsServed,
  MOD((s.releaseDate - s.draftDate), 30) AS daysServed
FROM SOLDIERS s
JOIN COMMANDER c ON s.sID = c.cID
```

```sql
ORDER BY (s.releaseDate - s.draftDate) DESC
FETCH FIRST 10 ROWS ONLY;
```

התוצאה שתתקבל מהרצת השאילתה היא

	commandername text	draftdate date	releasedate date	yearsserved double precision	monthsserved double precision	daysserved integer
1	Dorry Pleasure	2006-01-05	2009-12-12	3	11	27
2	Lena Newman	2006-01-24	2009-12-30	3	11	26
3	Bruce Duncan	2006-01-22	2009-12-16	3	10	14
4	Elizabeth Wiest	2006-01-04	2009-11-28	3	10	14
5	Gladys Gordon	2006-01-15	2009-11-26	3	10	1
6	Thin Furay	2006-01-14	2009-11-24	3	10	0
7	Martin Ifans	2006-01-14	2009-11-22	3	10	28
8	Lorraine Reeve	2006-02-27	2009-12-30	3	10	22
9	Nicolas Wiedlin	2006-02-01	2009-12-02	3	10	20
10	Randy Malone	2006-01-07	2009-11-06	3	10	19

Total rows: 10 Query complete 00:00:00.094

שאילתה 6 : רשימת מפקדים של יחידות שהשתתפו בלפחות 3 משימות, כולל שם המפקד, מזהה יחידה, מספר המשימות.

```sql
SELECT
  s.firstName,
  s.lastName,
  u.unID,
  COUNT(p.mID) AS missionCount
FROM COMMANDER c
JOIN SOLDIERS s ON c.cID = s.sID
JOIN UNIT u ON u.cID = c.cID
JOIN participates p ON u.unID = p.unID
GROUP BY s.firstName, s.lastName, u.unID
HAVING COUNT(p.mID) >= 3
ORDER BY missionCount DESC;
```

התוצאה שתתקבל מהרצת השאילתה היא

	firstname character varying (20)	lastname character varying (20)	unid numeric (9)	missioncount bigint
1	Regina	Matthau	129	8
2	Helen	Ruiz	54	7
3	Ryan	Aniston	232	7
4	Angelina	Heald	49	7
5	Rhett	Biehn	135	7
6	Kieran	Nash	20	7
7	Wade	Peet	44	6
8	Ernest	Pollak	173	6
9	Taylor	Armstrong	115	6
10	Stephanie	Crudup	157	6
11	Elijah	Lonsdale	51	6
12	Lorraine	Reeve	60	6
13	Crispin	Tanon	11	6
14	Hex	Cromwell	125	6
15	Cheech	McGoohan	158	6
16	Hookah	Patrick	204	6

Total rows: 129 Query complete 00:00:00.064

שאילתה 7 : כמה חיילים משרתים תחת כל מפקד, כולל שם המפקד ומספר החיילים, ממויין לפי מספר החיילים.

```
SELECT
  s.firstName AS commanderFirstName,
  s.lastName AS commanderLastName,
  COUNT(cm.crID) AS soldiersUnderCommand
FROM COMMANDER c
JOIN SOLDIERS s ON c.cID = s.sID
JOIN CREWMATE cm ON c.cID = cm.cID
GROUP BY s.firstName, s.lastName
ORDER BY soldiersUnderCommand DESC;
```

התוצאה שתתקבל מהרצת השאילתה היא

	commanderfirstname character varying (20)	commanderlastname character varying (20)	soldiersundercommand bigint
1	Hilary	Mohr	3
2	Darius	Baranski	3
3	Bo	Heslov	3
4	Naomi	Webb	3
5	Davy	Janney	3
6	Dylan	Curtis	3
7	Hector	Hanley	3
8	Matthew	Kattan	3
9	Howie	Coltrane	3
10	Gailard	Jenkins	3
11	Dick	Sutherland	3
12	Robbie	Crewson	3
13	Vanessa	Phillippe	3
14	Brothers	Cummings	3
15	James	Rankin	3
16	Joanna	Vega	3

Total rows: 466   Query complete 00:00:00.087

שאילתה 8 : עשרת המפקדים שיחידתם השתתפה בהכי הרבה משימות לצורך הענקת עיטור העוז.

```
SELECT
  s.firstName || ' ' || s.lastName AS commanderName,
  u.uName AS unitName,
  COUNT(p.mID) AS missionCount
FROM COMMANDER c
JOIN SOLDIERS s ON c.cID = s.sID
JOIN UNIT u ON c.cID = u.cID
JOIN participates p ON u.unID = p.unID
GROUP BY s.firstName, s.lastName, u.uName
ORDER BY missionCount DESC
FETCH FIRST 10 ROWS ONLY;
```

התוצאה שתתקבל מהרצת השאילתה היא

	commandername text	unitname character varying (20)	missioncount bigint
1	Regina Matthau	Iron tails 4	8
2	Angelina Heald	Bnei Or 2th	7
3	Ryan Aniston	Rays of fire 6	7
4	Kieran Nash	Galloping Horse	7
5	Rhett Biehn	Machatz 4	7
6	Helen Ruiz	Yiftach 2th	7
7	Elijah Lonsdale	Machatz 2th	6
8	Taylor Armstrong	Stars of fire 3	6
9	Crispin Tanon	Iron Fist	6
10	Cheech McGoohan	Netivei Esh 4	6

Total rows: 10 Query complete 00:00:00.057

3 שאילתות delete:

שאילתה 1 : בעקבות תפקודה המופתי של יחידה 101 בעזה, הוחלט בדרג הפיקודי העליון על הפיכת היחידה ליחידה מיוחדת ומסווגת ועל כן, יש צורך במחיקת כל הרישומים המעידים על קיום היחידה.

```
DELETE FROM participates p
WHERE p.UNID = 101;
```

Data Output Messages Notifications

	mid [PK] numeric (9)	unid [PK] numeric (9)
1	1	60
2	1	202
3	2	81
4	2	107
5	3	128
6	3	224
7	4	117
8	4	202
9	4	220
10	5	54

נמחקו נתוני היחידה 101.

לאחר הרצת פקודת rollback הנתונים יחזרו להיות כמו שהם היו בתמונה הראשונה.

שאילתה 2 : יחידה 5 התפרקה עקב בעיות משמעת, נמחק את חברי הצוות בטנקים של היחידה.

```
DELETE FROM CREWMATE
WHERE crID IN (
  SELECT cr.crID
  FROM CREWMATE cr
  JOIN COMMANDER c ON cr.cID = c.cID
  JOIN UNIT u ON u.cID = c.cID
  WHERE u.unID = 5
);
rollback;
```

```sql
DELETE FROM CREWMATE
WHERE crID IN (
  SELECT cr.crID
  FROM CREWMATE cr
  JOIN COMMANDER c ON cr.cID = c.cID
  JOIN UNIT u ON u.cID = c.cID
  WHERE u.unID =5
);
```

Data Output Messages Notifications

```
DELETE 3

Query returned successfully in 72 msec.
```

לאחר הרצת פקודת rollback הנתונים יחזרו להיות כמו שהם היו בתמונה הראשונה.

שאילתה 3 ושימוש בפקודת rollback:

הוחלט לשחרר משירות מילואים את כל החיילים שאינם בשירות פעיל כבר יותר מ15 שנה ושאינם מפקדים או נהגים. נמחק אותם מבסיס הנתונים.

```sql
begin;
DELETE FROM CREWMATE
WHERE crID IN (
    SELECT sID
    FROM SOLDIERS
    WHERE sID NOT IN (SELECT cID FROM COMMANDER)
      AND sID NOT IN (
          SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
      )
      AND releaseDate < CURRENT_DATE - INTERVAL '15 years'
);
DELETE FROM SOLDIERS
WHERE sID NOT IN (SELECT cID FROM COMMANDER)
  AND sID NOT IN (
      SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
  )
  AND releaseDate < CURRENT_DATE - INTERVAL '15 years';
rollback;
```

```sql
begin;
DELETE FROM CREWMATE
WHERE crID IN (
    SELECT sID
    FROM SOLDIERS
    WHERE sID NOT IN (SELECT cID FROM COMMANDER)
      AND sID NOT IN (
          SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
      )
      AND releaseDate < CURRENT_DATE - INTERVAL '15 years'
);
DELETE FROM SOLDIERS
WHERE sID NOT IN (SELECT cID FROM COMMANDER)
  AND sID NOT IN (
      SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
  )
  AND releaseDate < CURRENT_DATE - INTERVAL '15 years';
rollback;
```

Data Output Messages Notifications

```
WARNING:  there is already a transaction in progress
ROLLBACK

Query returned successfully in 113 msec.
```

type character varying (20)	crid [PK] numeric (9)	cid numeric (9)
Loader	601	1
Gunner	602	1
Driver	603	1
Loader	604	2
Gunner	605	2
Driver	606	2
Loader	607	3
Gunner	608	3
Driver	609	3
Loader	610	4
Gunner	611	4
Driver	612	4

לאחר הרצת פקודת המחיקה עם שימוש בפקודת rollback בסיס הנתונים נשמר.

```sql
DELETE FROM CREWMATE
WHERE crID IN (
    SELECT sID
```

```sql
    FROM SOLDIERS
    WHERE sID NOT IN (SELECT cID FROM COMMANDER)
      AND sID NOT IN (
          SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
      )
      AND releaseDate < CURRENT_DATE - INTERVAL '15 years'
);
DELETE FROM SOLDIERS
WHERE sID NOT IN (SELECT cID FROM COMMANDER)
  AND sID NOT IN (
      SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
  )
  AND releaseDate < CURRENT_DATE - INTERVAL '15 years';
```

Query Query History

```sql
DELETE FROM CREWMATE
WHERE crID IN (
    SELECT sID
    FROM SOLDIERS
    WHERE sID NOT IN (SELECT cID FROM COMMANDER)
      AND sID NOT IN (
          SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
      )
      AND releaseDate < CURRENT_DATE - INTERVAL '15 years'
);
DELETE FROM SOLDIERS
WHERE sID NOT IN (SELECT cID FROM COMMANDER)
  AND sID NOT IN (
      SELECT crID FROM CREWMATE WHERE LOWER(type) = 'driver'
  )
  AND releaseDate < CURRENT_DATE - INTERVAL '15 years';
```

Data Output Messages Notifications

DELETE 935

לאחר הרצת פקודת המחיקה ללא rollback נמחקו 935 חיילים מבסיס הנתונים.

3 שאילתות update:

שאילתה 1 : שינוי שם לinactive Unit ליחידות שאינן פעילות.

```sql
UPDATE UNIT
SET uName = 'Inactive Unit'
WHERE unID NOT IN (
    SELECT p.unID
    FROM participates p
    WHERE p.mID IN (
        SELECT m.mID
        FROM MISSION m
        WHERE m.mdate >= CURRENT_DATE + INTERVAL '-12 months'
    )
);
```

Query Query History

```sql
UPDATE UNIT
SET uName = 'Inactive Unit'
WHERE unID NOT IN (
    SELECT p.unID
    FROM participates p
    WHERE p.mID IN (
        SELECT m.mID
        FROM MISSION m
        WHERE m.mdate >= CURRENT_DATE + INTERVAL '-12 months'
    )
);
```

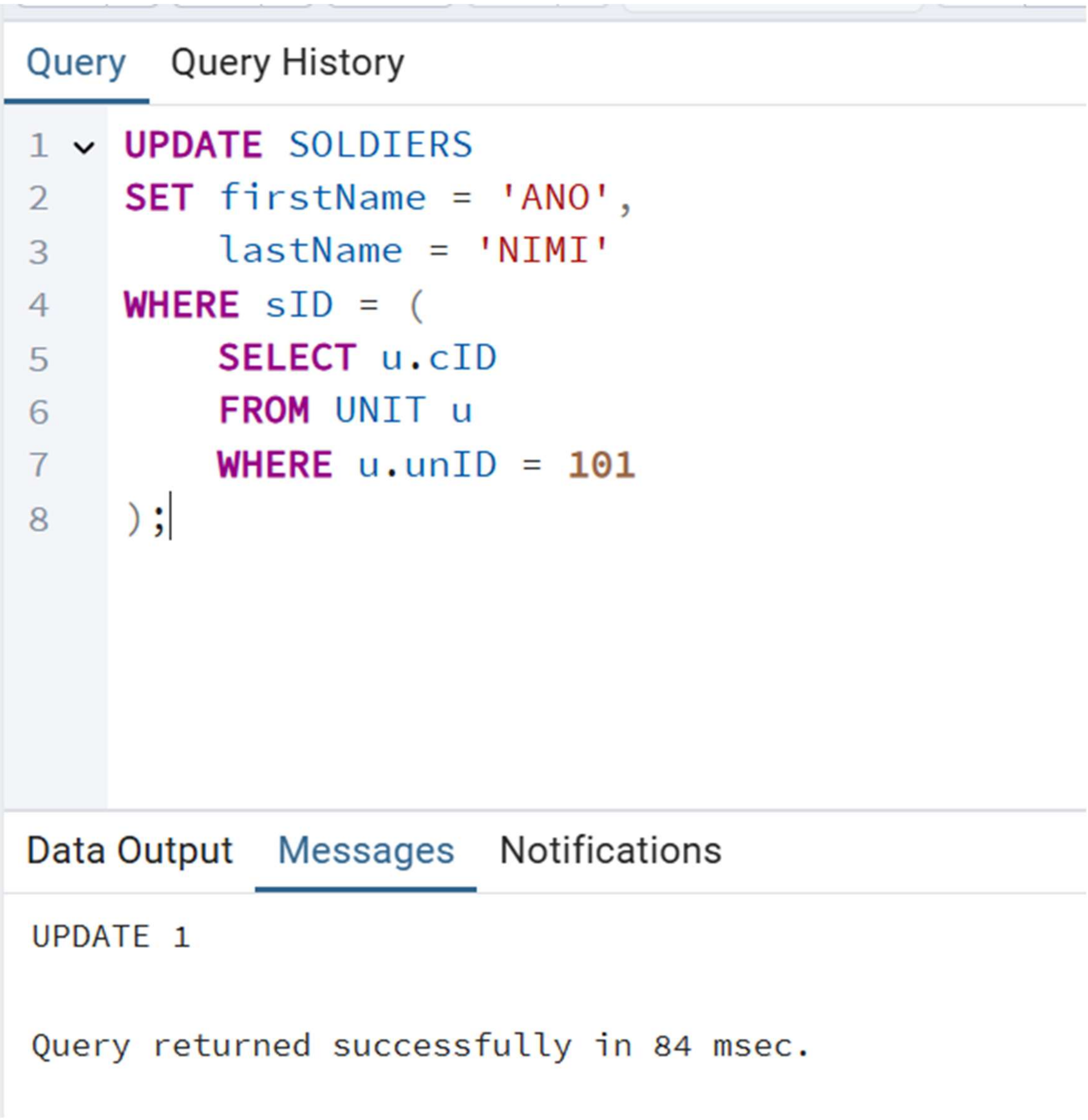
Data Output Messages Notifications

```
UPDATE 244

Query returned successfully in 76 msec.
```

לאחר הרצת פקודת rollback הנתונים יחזרו להיות כמו שהם היו בתמונה הראשונה.

שאילתה 2 : לאחר שיחידה 101 הפכה למסווגת, גם המפקד שלה נדרש לרדת לצללים ועל כן הוא שינה את שמו ל"אנו נימי"

```sql
UPDATE SOLDIERS
SET firstName = 'ANO',
    lastName = 'NIMI'
WHERE sID = (
    SELECT u.cID
    FROM UNIT u
    WHERE u.unID = 101
);
```



לאחר הרצת פקודת rollback הנתונים יחזרו להיות כמו שהם היו בתמונה הראשונה.

שאילתה 3: עדכון תאריך שחרור (releaseDate) לכל חייל שהוא מפקד, ששירת עד כה פחות משנתיים וחצי – תוספת של שנה לשחרור.

```
UPDATE SOLDIERS s
SET releaseDate = releaseDate + INTERVAL '12 months'
WHERE sID IN (
  SELECT s2.sID
  FROM SOLDIERS s2
  JOIN COMMANDER c ON s2.sID = c.cID
  WHERE s2.releaseDate - s2.draftDate < 912
);
```

Query | Query History

```sql
UPDATE SOLDIERS s
SET releaseDate = releaseDate + INTERVAL '12 months'
WHERE sID IN (
  SELECT s2.sID
  FROM SOLDIERS s2
  JOIN COMMANDER c ON s2.sID = c.cID
  WHERE s2.releaseDate - s2.draftDate < 912
);
```

Data Output | Messages | Notifications

```
UPDATE 79

Query returned successfully in 60 msec.
```

שאילתת Update עם Commit:

Query | Query History

```sql
begin;
UPDATE SOLDIERS s
SET releaseDate = releaseDate + INTERVAL '12 months'
WHERE sID IN (
  SELECT s2.sID
  FROM SOLDIERS s2
  JOIN COMMANDER c ON s2.sID = c.cID
  WHERE s2.releaseDate - s2.draftDate < 912
);
commit;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 48 msec.
```

לפני השינוי:

221	221	Wade	McDormand	2006-06-18	2009-11-01
222	222	Tcheky	Hiatt	2006-03-26	2009-10-12
223	223	Saffron	May	2006-04-22	2009-08-04
224	224	Gordon	Turner	2006-11-08	2009-01-26
225	225	Zooey	Imperioli	2006-05-24	2009-12-05
226	226	Suzi	Giraldo	2006-04-23	2009-09-14
227	227	Debbie	Swank	2006-06-30	2009-12-18
228	228	Fairuza	McCormack	2006-01-20	2009-02-22
229	229	Lenny	Seagal	2006-06-10	2009-05-23
230	230	Robin	Phoenix	2006-07-24	2009-03-22
231	231	Javon	Shearer	2006-08-10	2009-12-10
232	232	Janice	D'Onofrio	2006-09-18	2009-02-06
233	233	Lena	Rucker	2006-09-17	2009-05-25
234	234	Teri	McCain	2006-07-05	2009-11-06
235	235	Mika	Holly	2006-12-08	2009-02-27
236	236	Philip	Dushku	2006-01-28	2009-07-10

אחרי השינוי:

Data Output Messages Notifications

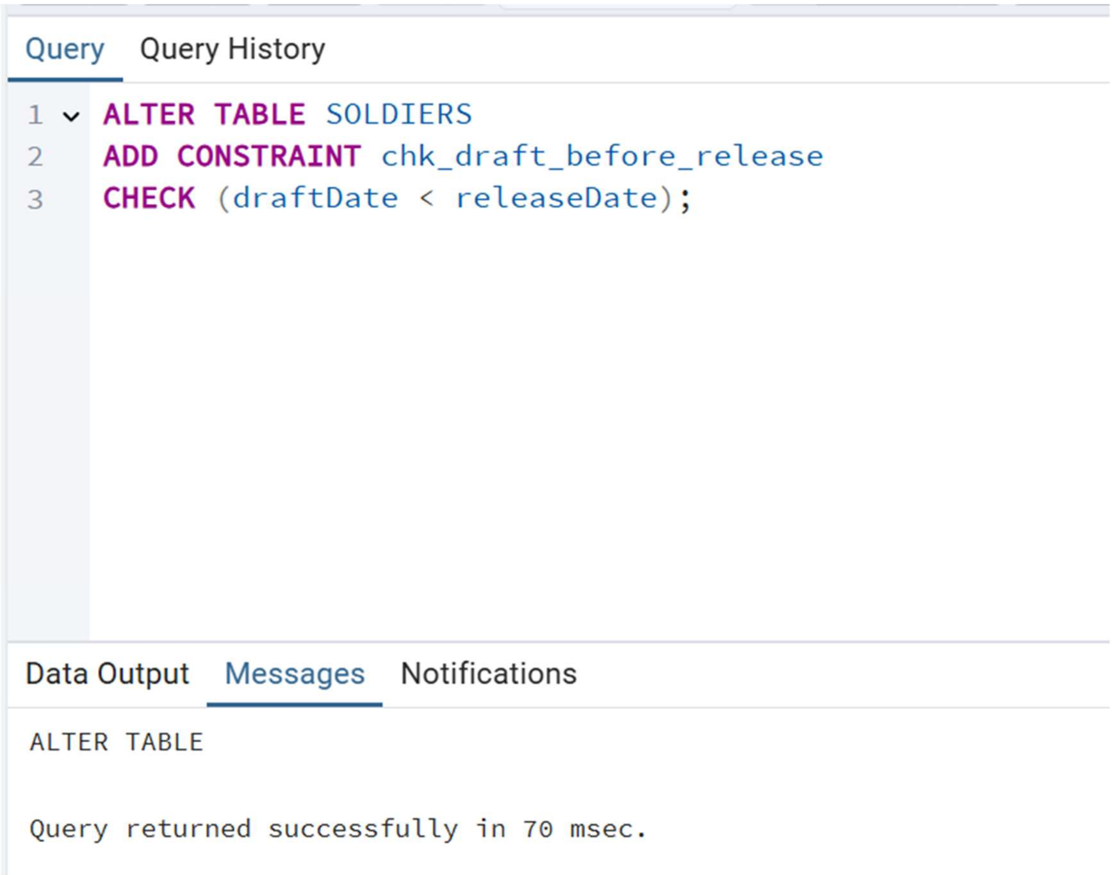
Showing rows: 1 to 1000 Page No:

	sid [PK] numeric (9)	firstname character varying (20)	lastname character varying (20)	draftdate date	releasedate date
221	221	Wade	McDormand	2006-06-18	2009-11-01
222	222	Tcheky	Hiatt	2006-03-26	2009-10-12
223	223	Saffron	May	2006-04-22	2009-08-04
224	224	Gordon	Turner	2006-11-08	2010-01-26
225	225	Zooey	Imperioli	2006-05-24	2009-12-05
226	226	Suzi	Giraldo	2006-04-23	2009-09-14
227	227	Debbie	Swank	2006-06-30	2009-12-18
228	228	Fairuza	McCormack	2006-01-20	2009-02-22
229	229	Lenny	Seagal	2006-06-10	2009-05-23
230	230	Robin	Phoenix	2006-07-24	2009-03-22
231	231	Javon	Shearer	2006-08-10	2009-12-10
232	232	Janice	D'Onofrio	2006-09-18	2010-02-06
233	233	Lena	Rucker	2006-09-17	2009-05-25
234	234	Teri	McCain	2006-07-05	2009-11-06
235	235	Mika	Holly	2006-12-08	2010-02-27
236	236	Philip	Dushku	2006-01-28	2009-07-10

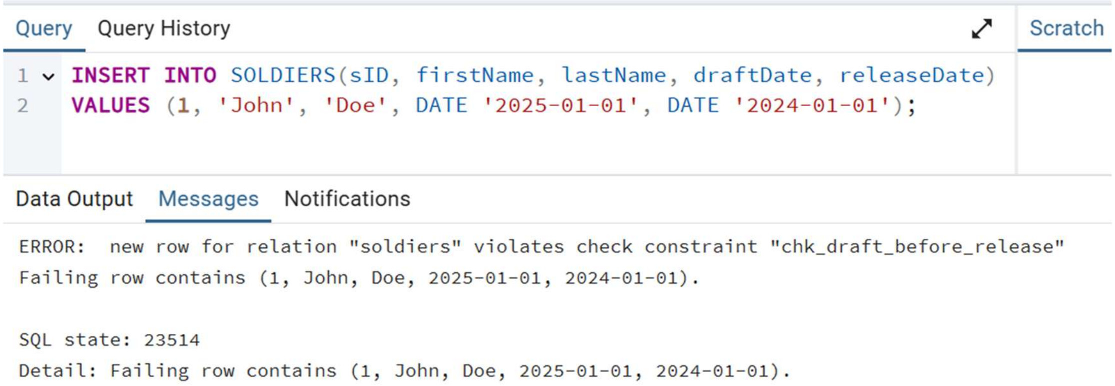
אילוצים

אילוץ 1 : האילוץ מכריח כי לכל חייל תאריך השחרור חייב להיות אחרי תאריך הגיוס

```
ALTER TABLE SOLDIERS
ADD CONSTRAINT chk_draft_before_release
CHECK (draftDate < releaseDate);
```



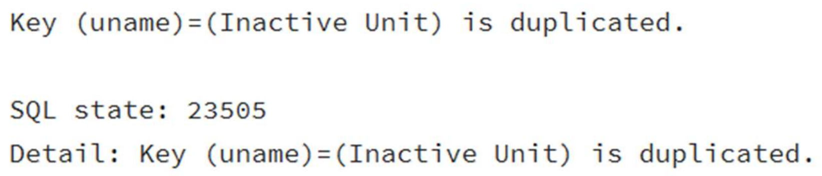
ננסה לבצע פקודה שמפרה את האילוץ



אילוץ 2 : האילוץ מחייב שלכל יחידה יהיה שם מיוחד רק לה

```
ALTER TABLE UNIT
ADD CONSTRAINT unique_unit_name
UNIQUE (uName);
```

ננסה לבצע פקודה שמפרה את האילוץ



אילוץ 3 : האילוץ מכריח שלכל איש צוות בטנק חייב להיות תפקיד

```
ALTER TABLE CREWMATE
ALTER COLUMN type SET NOT NULL;
```

Query Query History

```
ALTER TABLE CREWMATE
ALTER COLUMN type SET NOT NULL;
```

Data Output Messages Notifications

```
ALTER TABLE

Query returned successfully in 54 msec.
```

ננסה לבצע פקודה שמפרה את האילוץ

Query Query History

```
INSERT INTO CREWMATE (type, crID, cID)
VALUES (NULL, 10, 100);
```

Data Output Messages Notifications

```
ERROR:  null value in column "type" of relation "crewmate" violates not-null constraint
Failing row contains (null, 10, 100).

SQL state: 23502
Detail: Failing row contains (null, 10, 100).
```

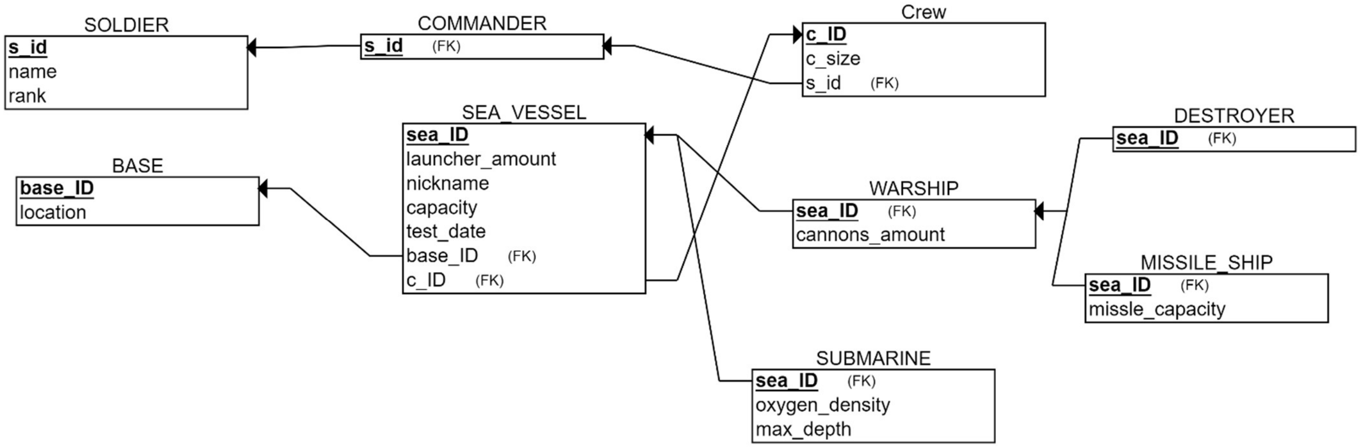
שלב 3 – אינטגרציה ומבטים

עשינו אינטגרציה עם הקבוצה של אלון ובעז שבחרו פרויקט בנושא דומה - חיל הים.

עיצוב DSD וERD חדשים

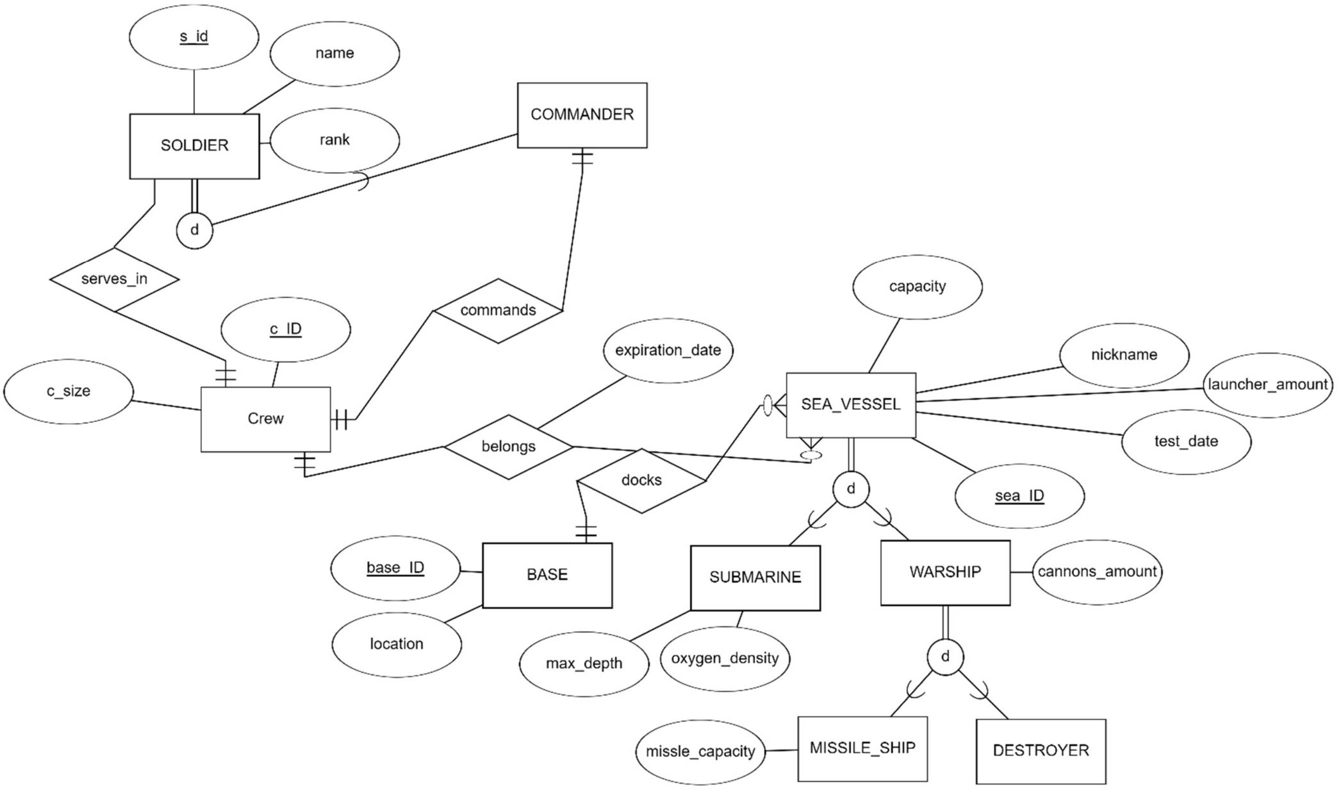
עיצוב DSD של הקבוצה השנייה

לאחר שהתבוננו ב-SQL שלהם והבנו את התלויות והמפתחות הזרים, יצרנו DSD.



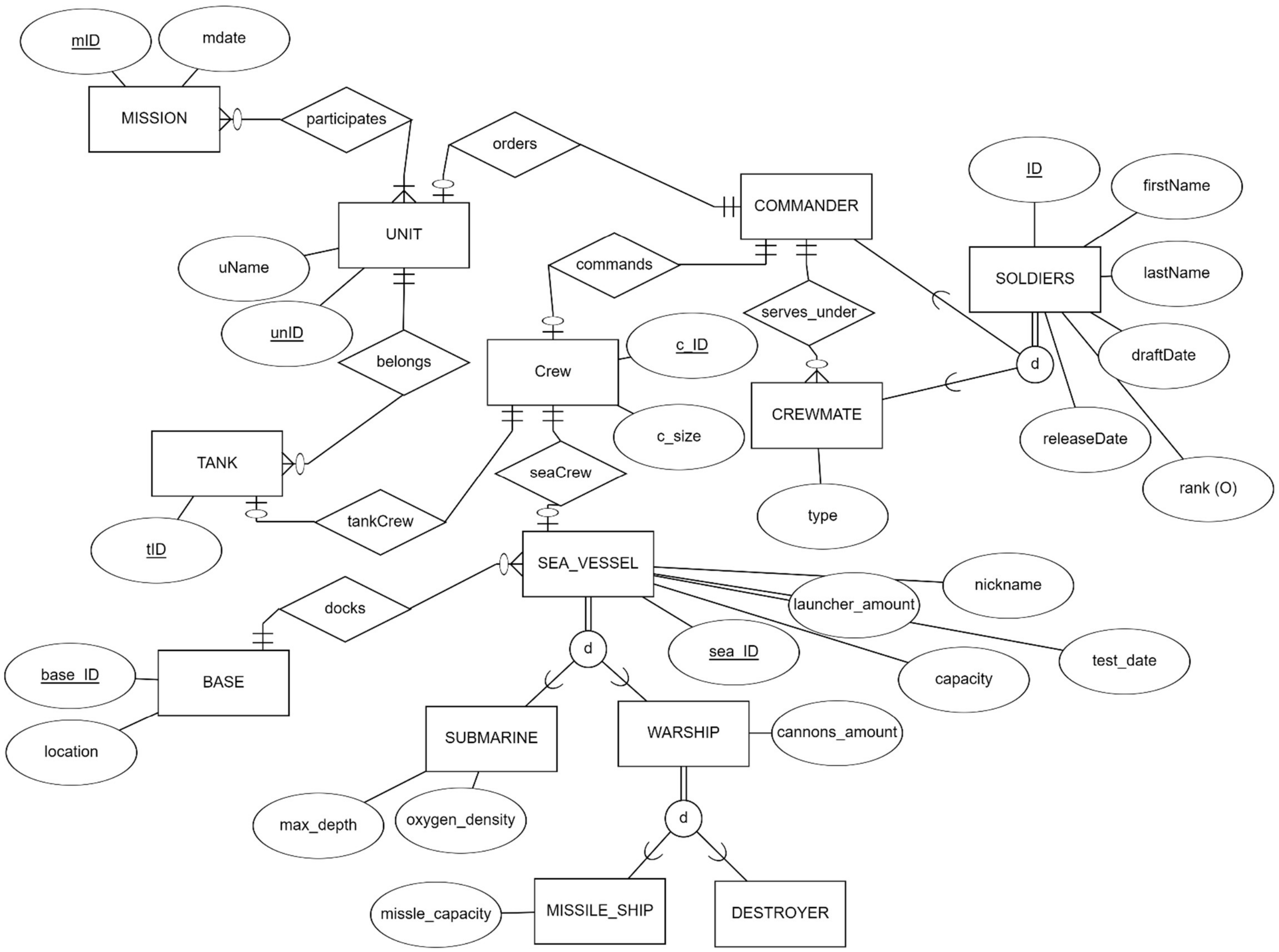
עיצוב ERD של הקבוצה השנייה

מתוך הDSD שיצרנו, אנחנו נייצר את ה ERD-של בסיס הנתונים. בעזרת הינדוס לאחור (reverse engineering) .

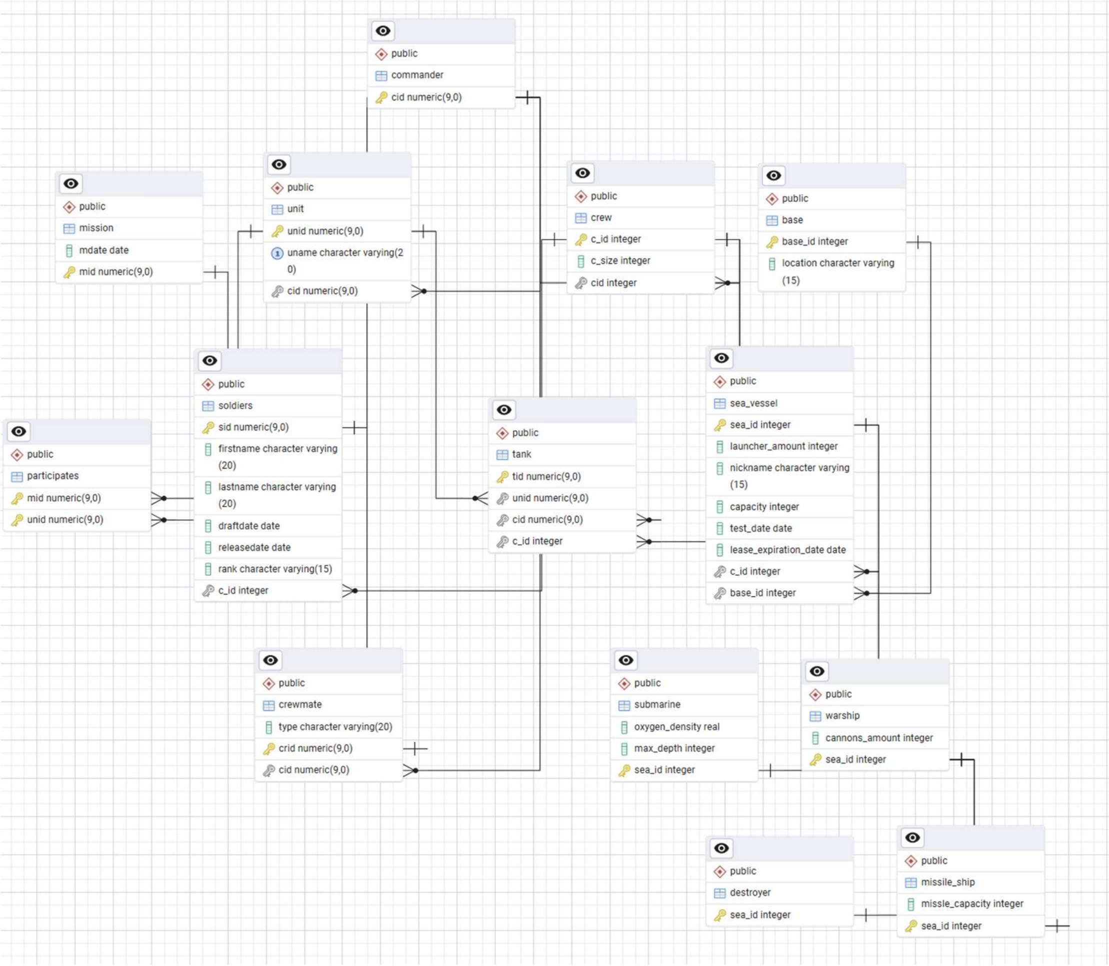


עיצובי ERD ו- DSD משולבים של שתי הקבוצות

לאחר מחשבה מעמיקה, הבנו שהדרך הנכונה לשלב בין שני תרשימי ה ERD-היא לחבר את סכמת הSoldiers שלנו לסכמת הSoldier של הקבוצה השנייה.
אותו הדבר נעשה לגבי סכמת הCommander.
לאחר מכן, יצרנו סכמה חדשה Crew, אליה פונות הן הסכמות שלנו, כמו סכמת ה Tank והן הסכמות של הקבוצה השנייה, כמו כל כלי הים.



נייצא את טבלת ה- dsd המשותפת מתוך ה – pgAdmin.


public
commander
cid numeric(9,0)
public
mission
mdate date
mid numeric(9,0)
public
unit
unid numeric(9,0)
uname character varying(20)
cid numeric(9,0)
public
crew
c_id integer
c_size integer
cid integer
public
base
base_id integer
location character varying (15)
public
participates
mid numeric(9,0)
unid numeric(9,0)
public
soldiers
sid numeric(9,0)
firstname character varying (20)
lastname character varying (20)
draftdate date
releasedate date
rank character varying(15)
c_id integer
public
tank
tid numeric(9,0)
unid numeric(9,0)
cid numeric(9,0)
c_id integer
public
sea_vessel
sea_id integer
launcher_amount integer
nickname character varying (15)
capacity integer
test_date date
lease_expiration_date date
c_id integer
base_id integer
public
crewmate
type character varying(20)
crid numeric(9,0)
cid numeric(9,0)
public
submarine
oxygen_density real
max_depth integer
sea_id integer
public
warship
cannons_amount integer
sea_id integer
public
destroyer
sea_id integer
public
missile_ship
missle_capacity integer
sea_id integer

עדכון בסיס הנתונים

הסקת הסכמות החדשות מתוך הERD המשולב

השינויים בסכמות המידע מתחלקים לשני חלקים :

החלק הראשון מתייחס לטבלאות החדשות של הקבוצה השנייה, ובשבילן ניצור טבלאות חדשות באמצעות פקודת CREATE .

```
-- Table for crews
CREATE TABLE Crew (
  c_ID INTEGER PRIMARY KEY,
  c_size INTEGER NOT NULL,
  cID INTEGER NOT NULL,
  FOREIGN KEY (cID) REFERENCES Commander(cID)
);

-- Table for military bases
CREATE TABLE Base (
  base_ID INTEGER PRIMARY KEY,
  location VARCHAR(15) NOT NULL
);

-- Table for general sea vessels
CREATE TABLE Sea_Vessel (
  sea_ID INTEGER PRIMARY KEY,
  launcher_amount INTEGER NOT NULL,
  nickname VARCHAR(15) NOT NULL,
  capacity INTEGER NOT NULL,
  test_date DATE NOT NULL,
  lease_expiration_date DATE NOT NULL,
  c_ID INTEGER NOT NULL,
  base_ID INTEGER NOT NULL,
  FOREIGN KEY (c_ID) REFERENCES Crew(c_ID),
  FOREIGN KEY (base_ID) REFERENCES Base(base_ID)
);

-- Table for submarines
CREATE TABLE Submarine (
  oxygen_density REAL NOT NULL,
  max_depth INTEGER NOT NULL,
  sea_ID INTEGER PRIMARY KEY,
  FOREIGN KEY (sea_ID) REFERENCES Sea_Vessel(sea_ID)
);

-- Table for warships
CREATE TABLE Warship (
```

```
  cannons_amount INTEGER NOT NULL,
  sea_ID INTEGER PRIMARY KEY,
  FOREIGN KEY (sea_ID) REFERENCES Sea_Vessel(sea_ID)
);

-- Table for missile ships
CREATE TABLE Missile_Ship (
  missle_capacity INTEGER NOT NULL,
  sea_ID INTEGER PRIMARY KEY,
  FOREIGN KEY (sea_ID) REFERENCES Warship(sea_ID)
);

-- Table for destroyers
CREATE TABLE Destroyer (
  sea_ID INTEGER PRIMARY KEY,
  FOREIGN KEY (sea_ID) REFERENCES Warship(sea_ID)
);
```

החלק השני מתייחס לשינויים בטבלאות הקיימות במקרה שלנו Soldiers ו, Tank)- עבורם נשתמש בפקודת ALTER TABLE .

עבור חייל – נחליט ששם משפחה הוא לא מידע חיוני. ונקבע שחייל ללא תאריך גיוס ושחרור, תאריך הגיוס שלו יהיה היום, ותאריך השחרור בעוד שלוש שנים.
הוספנו גם תכונה חדשה בשם c_id שתהיה מפתח זר עבור הצוות של החייל

עבור טנק – נוסיף מפתח זר חדש של צוות. כאשר לכל צוות יהיה מפקד הצוות שהוא גם מפקד הטנק.

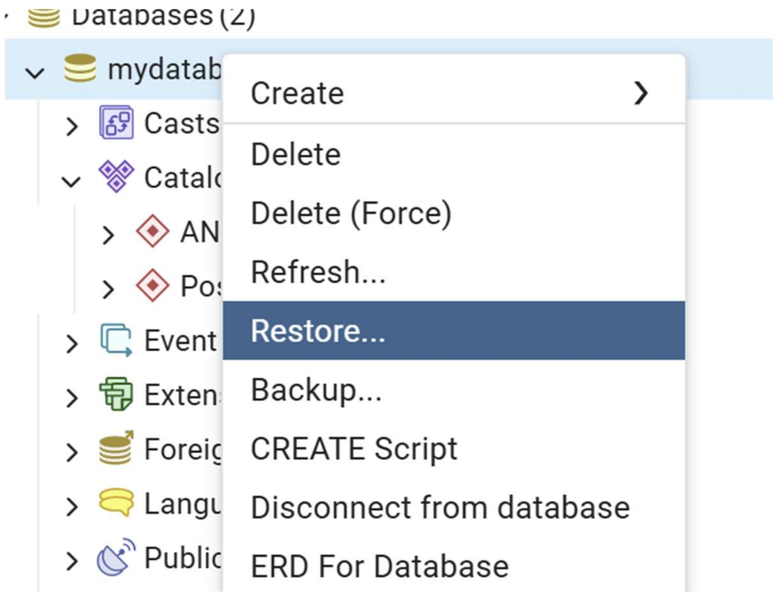
```
-- Add 'rank' column to Soldiers table
ALTER TABLE Soldiers ADD COLUMN rank VARCHAR(15);

-- Allow 'lastname' column to be nullable
ALTER TABLE Soldiers ALTER COLUMN lastname DROP NOT NULL;

-- Set default value for 'draftdate' to today's date
ALTER TABLE Soldiers ALTER COLUMN draftdate SET DEFAULT
CURRENT_DATE;

-- Set default value for 'releaseDate' to 3 years from
today's date
ALTER TABLE Soldiers ALTER COLUMN releaseDate SET DEFAULT
(CURRENT_DATE + INTERVAL '3 years');

-- Add 'c_ID' column to Soldiers if it doesn't already
exist
```

```
ALTER TABLE Soldiers ADD COLUMN c_ID INTEGER;

-- Add foreign key constraint to Soldiers referencing Crew
ALTER TABLE Soldiers ADD CONSTRAINT fk_soldiers_crew
FOREIGN KEY (c_ID) REFERENCES Crew(c_ID);

-- Add 'c_ID' column to Tank if it doesn't already exist
ALTER TABLE Tank ADD COLUMN c_ID INTEGER;

-- Add foreign key constraint to Tank referencing Crew
ALTER TABLE Tank ADD CONSTRAINT fk_tank_crew FOREIGN KEY
(c_ID) REFERENCES Crew(c_ID);
```

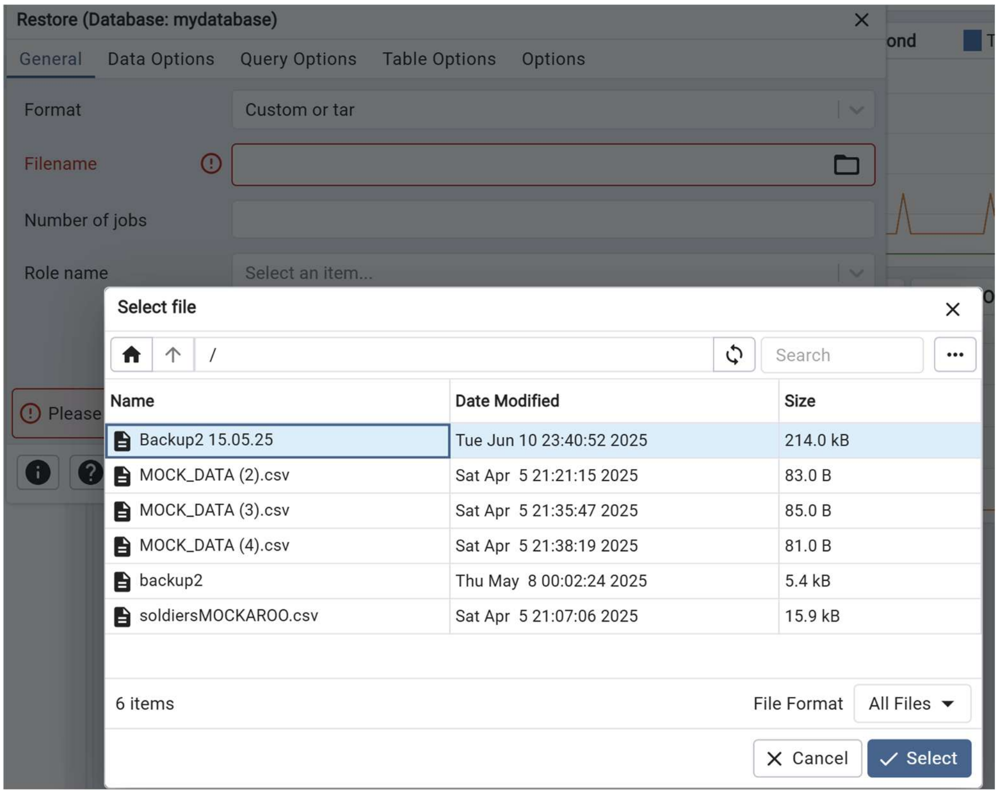
הכנסת נתונים לטבלאות

את הנתונים לטבלאות של הצוות השני הכנסנו בעזרת קובץ הגיבוי שלהם. כאשר את כל מספר הID שינינו (הוספנו 10,000) בשביל למנוע התנגשויות.

יצרנו גם 600 צוותים (Crew) חדשים בשביל הטנקים שקיימים במערכת שלנו.



נבחר את הקובץ ונעדכן.

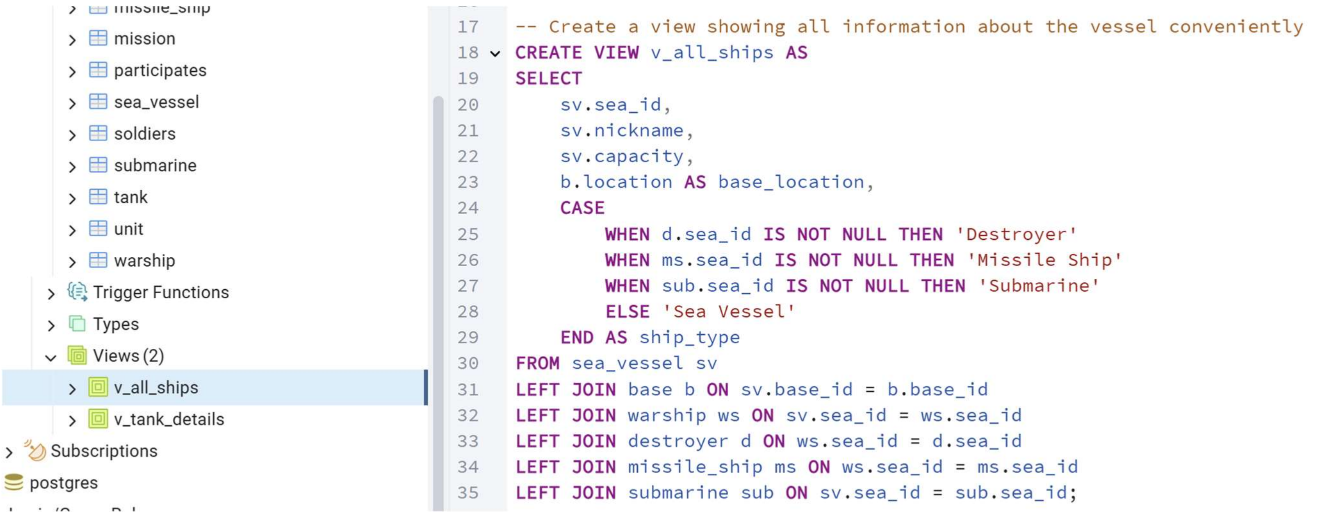

Restore (Database: mydatabase)
General
Data Options
Query Options
Table Options
Options
Format
Custom or tar
Filename
Number of jobs
Role name
Select an item...
Please
Select file
/
Search
Name
Date Modified
Size
Backup2 15.05.25
Tue Jun 10 23:40:52 2025
214.0 kB
MOCK_DATA (2).csv
Sat Apr 5 21:21:15 2025
83.0 B
MOCK_DATA (3).csv
Sat Apr 5 21:35:47 2025
85.0 B
MOCK_DATA (4).csv
Sat Apr 5 21:38:19 2025
81.0 B
backup2
Thu May 8 00:02:24 2025
5.4 kB
soldiersMOCKAROO.csv
Sat Apr 5 21:07:06 2025
15.9 kB
6 items
File Format
All Files
Cancel
Select

מבטים

יצירת 2 מבטים

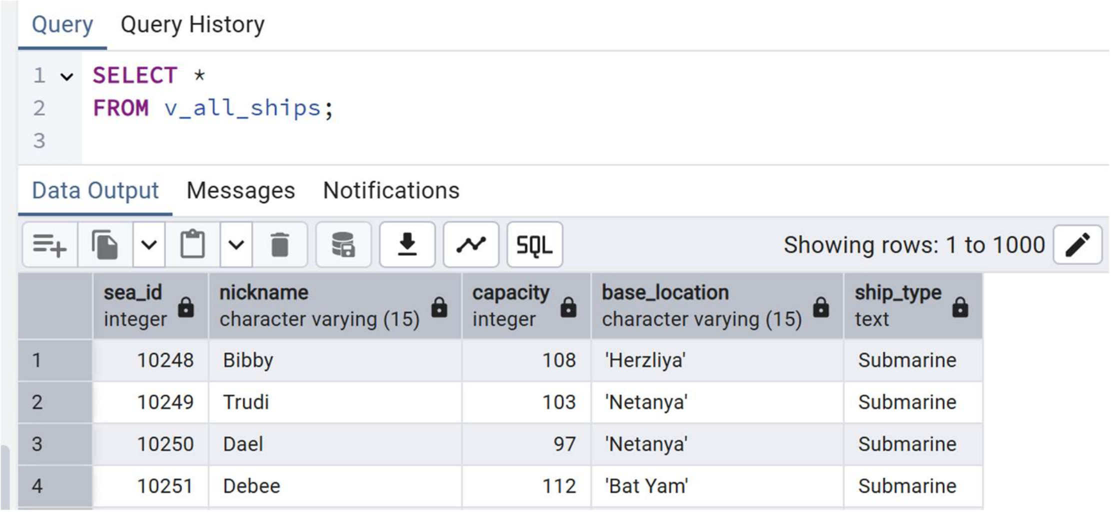
ניצור מבט עבור האגף של הקבוצה השנייה שייתן לנו את היכולת לראות את כל הספינות בצורה נוחה

```sql
-- Create a view showing all information about the vessel conveniently
CREATE VIEW v_all_ships AS
SELECT
    sv.sea_id,
    sv.nickname,
    sv.capacity,
    b.location AS base_location,
    CASE
        WHEN d.sea_id IS NOT NULL THEN 'Destroyer'
        WHEN ms.sea_id IS NOT NULL THEN 'Missile Ship'
        WHEN sub.sea_id IS NOT NULL THEN 'Submarine'
        ELSE 'Sea Vessel'
    END AS ship_type
FROM sea_vessel sv
LEFT JOIN base b ON sv.base_id = b.base_id
LEFT JOIN warship ws ON sv.sea_id = ws.sea_id
LEFT JOIN destroyer d ON ws.sea_id = d.sea_id
LEFT JOIN missile_ship ms ON ws.sea_id = ms.sea_id
LEFT JOIN submarine sub ON sv.sea_id = sub.sea_id;
```



נפעיל פקודת * select על המבט החדש שיצרנו ונראה את המידע על הספינות.

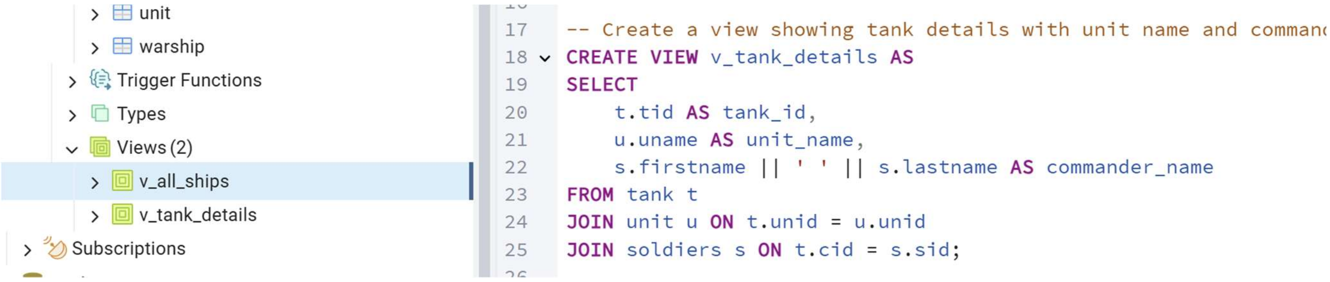
```sql
SELECT *
FROM v_all_ships;
```



	sea_id integer	nickname character varying (15)	capacity integer	base_location character varying (15)	ship_type text
1	10248	Bibby	108	'Herzliya'	Submarine
2	10249	Trudi	103	'Netanya'	Submarine
3	10250	Dael	97	'Netanya'	Submarine
4	10251	Debee	112	'Bat Yam'	Submarine

נוסיף מבט שני עבור האגף שלנו שיאפשר לראות את כל המידע החשוב על הטנק.

```sql
-- Create a view showing tank details with unit name and commander's full name
CREATE VIEW v_tank_details AS
SELECT
    t.tid AS tank_id,
    u.uname AS unit_name,
    s.firstname || ' ' || s.lastname AS commander_name
FROM tank t
JOIN unit u ON t.unid = u.unid
JOIN soldiers s ON t.cid = s.sid;
```



נריץ פקודת * select בשביל לראות את הטנקים ואת כל המידע החשוב עליהם

```sql
SELECT *
FROM v_tank_details;
```

```
SELECT *
FROM v_tank_details;
```

Data Output Messages Notifications

Showing rows: 1

	tank_id numeric (9)	unit_name character varying (20)	commander_name text
1	1	Barak	Jet Gore
2	2	Rays of fire 6	Latin Garber
3	3	Saar me-Golan 6	Sona Valentin
4	4	Eshet 5	Nicolas Alston
5	5	Galloping Horse 2	Candice Sheen

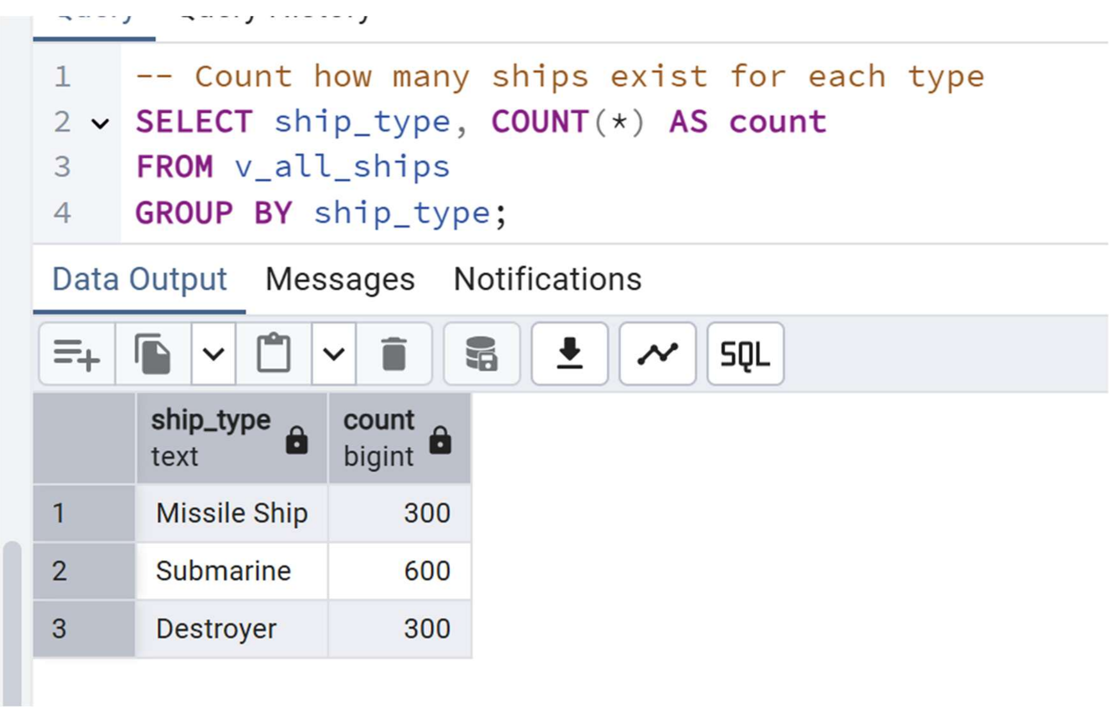
יצירת 2 שאילתות עבור כל מבט

ניצור 4 שאילתות על המערכת המשולבת, כאשר נשתמש במבטים שזה עתה יצרנו. המבטים יאפשרו לשאילתות להיות פשוטות ונוחות לקריאה בלי שימוש בJoinים.

שאילתה 1 עבור המבט V_All_Ships : השאילתה תספור ותחזיר את הכמות של כל סוג של ספינות שקיימות במאגר.

```
-- Count how many ships exist for each type
SELECT ship_type, COUNT(*) AS count
FROM v_all_ships
GROUP BY ship_type;
```

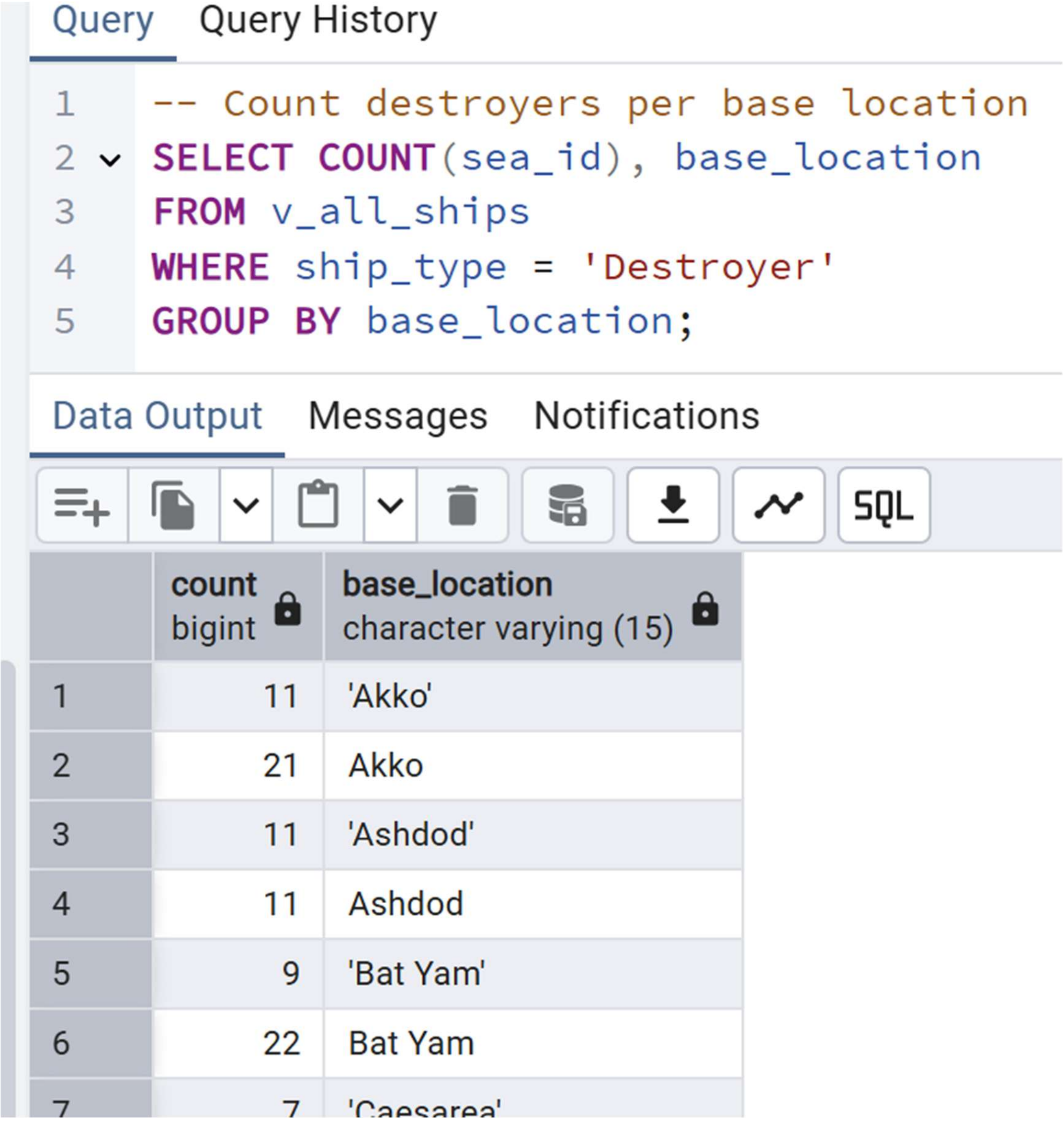
נריץ את השאילתה :



שאילתה 2 עבור המבט V_All_Ships : השאילתה תחזיר את כל הבסיסים שמשחתתות חונות באותו הבסיס. כאשר מספר המשחתות החונות יחושב ויוצג.

```
-- Count destroyers per base location
SELECT COUNT(sea_id), base_location
FROM v_all_ships
WHERE ship_type = 'Destroyer'
GROUP BY base_location;
```

נריץ את השאילתה :



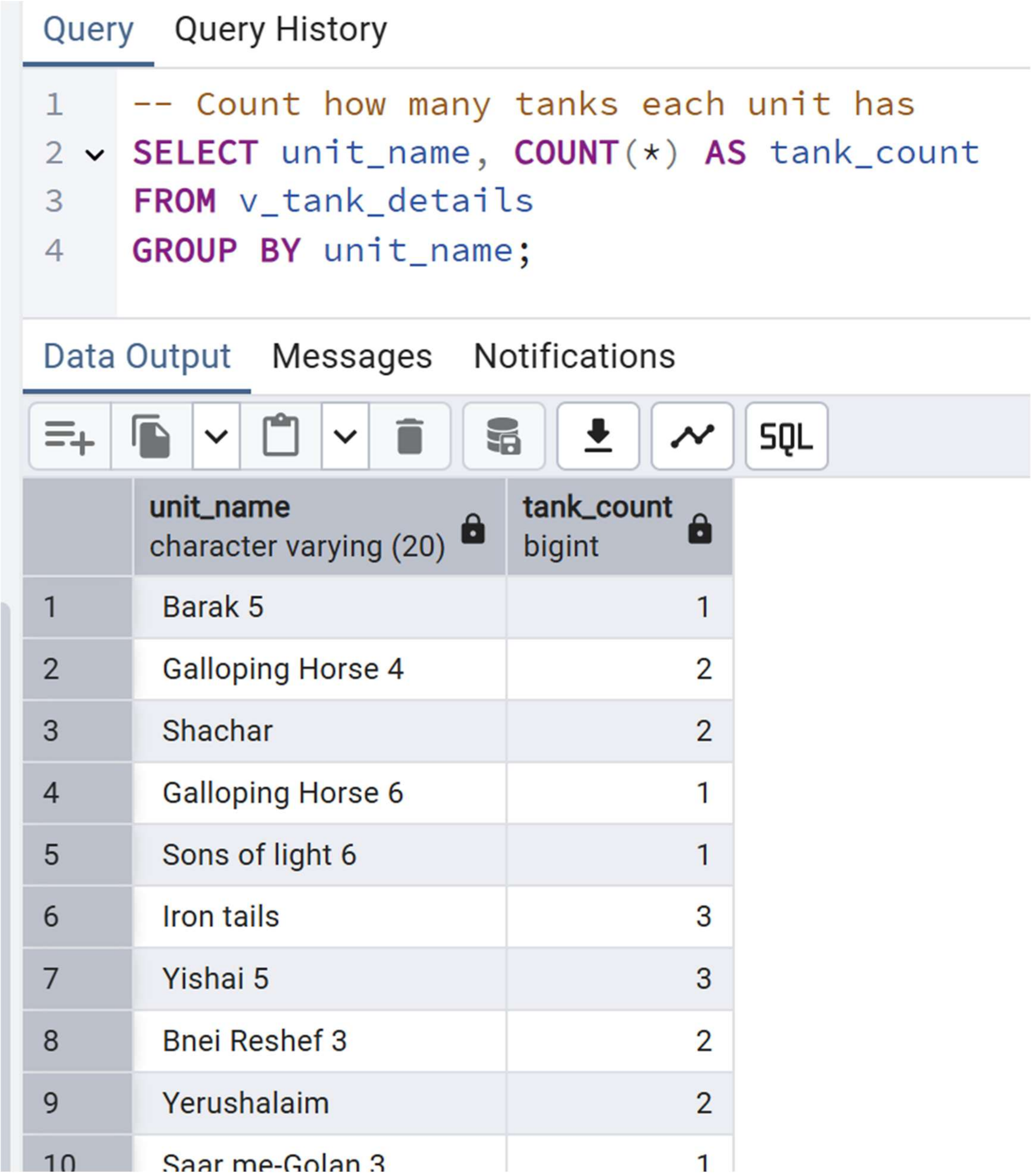

```sql
-- Count destroyers per base location
SELECT COUNT(sea_id), base_location
FROM v_all_ships
WHERE ship_type = 'Destroyer'
GROUP BY base_location;
```

	count bigint	base_location character varying (15)
1	11	'Akko'
2	21	Akko
3	11	'Ashdod'
4	11	Ashdod
5	9	'Bat Yam'
6	22	Bat Yam
7	7	'Caesarea'

שאילתה 1 עבור המבט V_Tank_Details : השאילתה תבדוק עבור כל יחידה כמה טנקים קיימים באותה יחידה, ותחזיר את המידע.

```sql
-- Count how many tanks each unit has
SELECT unit_name, COUNT(*) AS tank_count
FROM v_tank_details
GROUP BY unit_name;
```

נריץ את השאילתה :



Query　Query History

```sql
-- Count how many tanks each unit has
SELECT unit_name, COUNT(*) AS tank_count
FROM v_tank_details
GROUP BY unit_name;
```

Data Output　Messages　Notifications

	unit_name character varying (20)	tank_count bigint
1	Barak 5	1
2	Galloping Horse 4	2
3	Shachar	2
4	Galloping Horse 6	1
5	Sons of light 6	1
6	Iron tails	3
7	Yishai 5	3
8	Bnei Reshef 3	2
9	Yerushalaim	2
10	Saar me-Golan 3	1

שאילתה 2 עבור המבט V_Tank_Details : השאילתה תאפשר לעדכן את השם הפרטי ושם המשפחה של מפקד של טנק מסוים.

```sql
UPDATE soldiers
SET firstname = 'ahuvya', lastname = 'betsalel'
WHERE sid = (
    SELECT t.cid
    FROM tank t
    JOIN v_tank_details td ON t.tid = td.tank_id
    WHERE td.tank_id = 5
);
```

נראה את העדכון של השאילתה. כרגע השם הפרטי ושם המשפחה של טנק מספר 5 זה Candice Sheen.

```sql
SELECT commander_name
FROM v_tank_details
WHERE tank_id = 5;
```

Data Output Messages Notificati

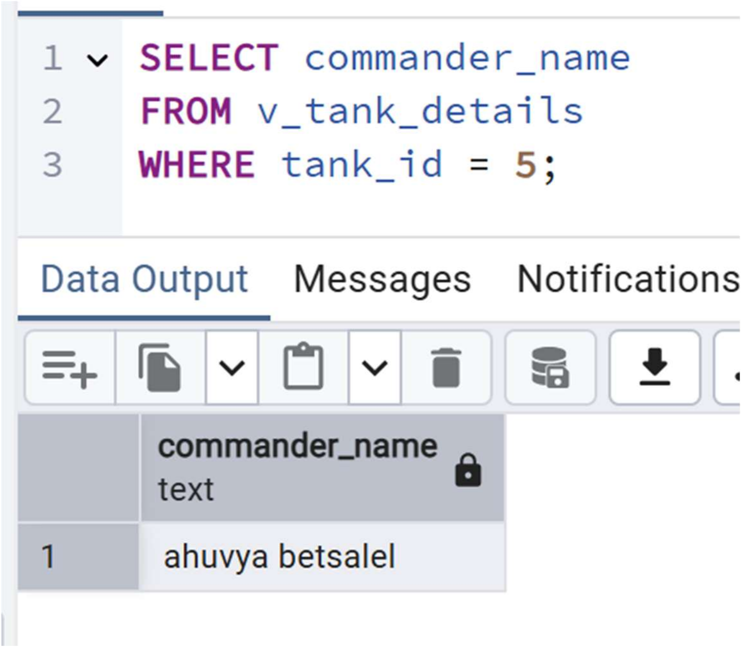
	commander_name text
1	Candice Sheen

אחרי הרצת השאילתה נבדוק וניווכח כי השם באמת השתנה ל John Doe

```sql
UPDATE soldiers
SET firstname = 'ahuvya', lastname = 'betsalel'
WHERE sid = (
    SELECT t.cid
    FROM tank t
    JOIN v_tank_details td ON t.tid = td.tank_id
    WHERE td.tank_id = 5
);
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 148 msec.
```


1 SELECT commander_name
2 FROM v_tank_details
3 WHERE tank_id = 5;
Data Output
Messages
Notifications
commander_name
text
1
ahuvya betsalel

שלב 4 – תכנות

פונקציות

פונקציה 1 - הפונקציה fn_commander_performance מחזירה מצביע (refcursor) עם דו"ח על מפקדים הכולל מספר טנקים, מספר משימות ורמת ניסיון לפי כמות המשימות.

```
-- Function returning a refcursor with commanders'
performance report:
-- number of tanks, missions, and experience level
based on missions count.
CREATE OR REPLACE FUNCTION fn_commander_performance()
RETURNS refcursor AS $$
DECLARE
    ref refcursor := 'my_cursor';
BEGIN
    OPEN ref FOR
    SELECT
        c.cid AS commander_id,
        COUNT(DISTINCT t.tid) AS num_tanks,
        COUNT(DISTINCT m.mid) AS num_missions,
        CASE
            WHEN COUNT(DISTINCT m.mid) >= 5 THEN
'Experienced'
            WHEN COUNT(DISTINCT m.mid) >= 2 THEN
'Intermediate'
            ELSE 'Newbie'
        END AS experience_level
    FROM
        commander c
        JOIN crew cr ON c.cid = cr.cid
        JOIN tank t ON t.cid = cr.cid
        JOIN unit u ON t.unid = u.unid
        JOIN participates p ON u.unid = p.unid
        JOIN mission m ON p.mid = m.mid
    GROUP BY c.cid;

    RETURN ref;
END;
$$ LANGUAGE plpgsql;
```

הפונקציה נוצרה בהצלחה:

- Functions (2)
  - fn_commander_performance()
  - fn_soldier_unit_summary(commander_id numeric)

הפונקציה רצה בהצלחה:

Query | Query History

```
SELECT fn_commander_performance();
FETCH ALL FROM my_cursor;
```

Data Output | Messages | Notifications

Showing rows: 1 to 340

	commander_id numeric (9)	num_tanks bigint	num_missions bigint	experience_level text
1	1	1	2	Intermediate
2	2	1	8	Experienced
3	3	1	3	Intermediate
4	4	1	3	Intermediate

פונקציה 2 - הפונקציה fn_soldier_unit_summary מקבלת מזהה מפקד ומחזירה טבלה עם רשימת החיילים בצוותים שתחתיו כולל שם היחידה, מזהה הטנק ותפקיד החייל בצוות.

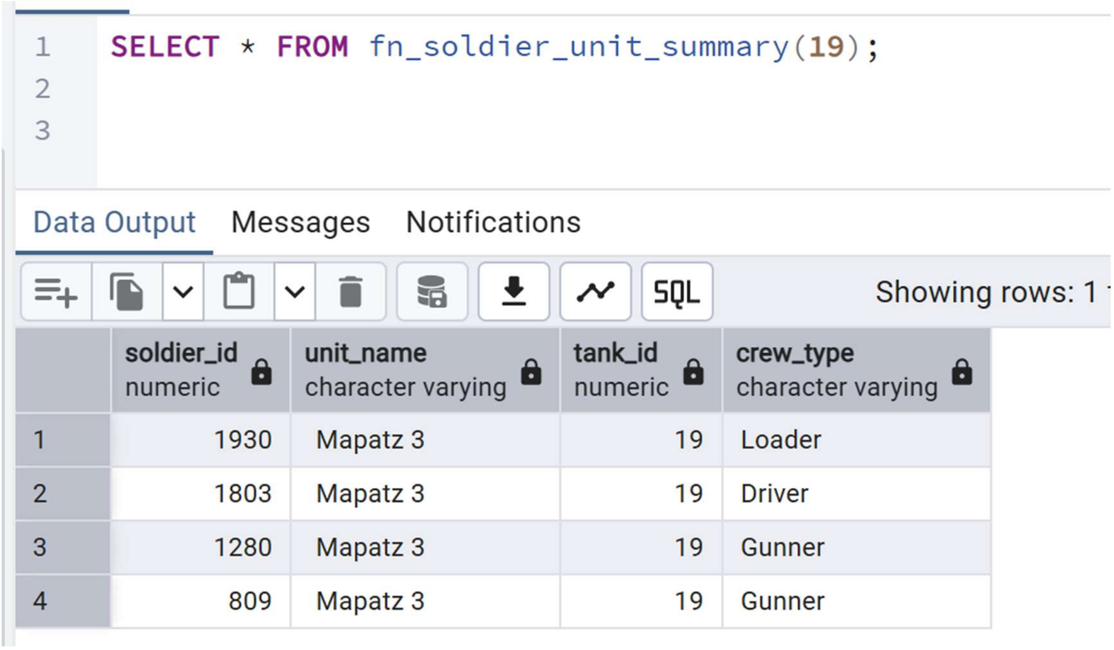
```
-- Function returning a soldier-unit summary for a specific commander,
-- including unit name, tank ID, and crew role.
CREATE OR REPLACE FUNCTION
fn_soldier_unit_summary(commander_id numeric)
RETURNS TABLE (
    soldier_id numeric,
    unit_name varchar,
    tank_id numeric,
    crew_type varchar
) AS $$
BEGIN
```

```sql
    RETURN QUERY
    SELECT
        s.sid AS soldier_id,
        u.uname AS unit_name,
        t.tid AS tank_id,
        cm.type AS crew_type
    FROM
        soldiers s
        JOIN crew cr ON s.c_id = cr.cid
        JOIN commander c ON cr.cid = c.cid
        JOIN tank t ON t.cid = cr.cid
        JOIN unit u ON t.unid = u.unid
        LEFT JOIN crewmate cm ON cm.crid = s.sid
    WHERE c.cid = commander_id;
END;
$$ LANGUAGE plpgsql;
```

הפונקציה נוצרה בהצלחה:

Functions (2)
- fn_commander_performance()
- fn_soldier_unit_summary(commander_id numeric)

הפונקציה רצה בהצלחה:




```sql
SELECT * FROM fn_soldier_unit_summary(19);
```

	soldier_id numeric	unit_name character varying	tank_id numeric	crew_type character varying
1	1930	Mapatz 3	19	Loader
2	1803	Mapatz 3	19	Driver
3	1280	Mapatz 3	19	Gunner
4	809	Mapatz 3	19	Gunner

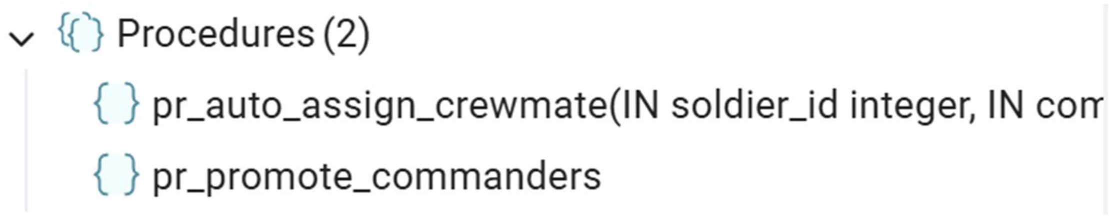
פרוצדורות

פרוצדורה 1 - הפרוצדורה pr_promote_commanders מקדמת מפקדים בדרגה לפי מספר המשימות שביצעו. 5 ומעלה ל – admiral, 3\4 ל – capitan, ואת השאר משאירה "sailor".

```sql
-- Procedure that promotes commanders based on number of missions:
-- commanders with 5+ missions become 'admiral', with 3-4 missions become 'captain', otherwise 'sailor'.

CREATE OR REPLACE PROCEDURE pr_promote_commanders()
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN
        SELECT s.sid AS soldier_id, COUNT(DISTINCT m.mid) AS missions
        FROM commander c
        JOIN crew cr ON c.cid = cr.cid
        JOIN soldiers s ON c.cid = s.sid
        JOIN tank t ON t.cid = cr.cid
```

```
        JOIN unit u ON t.unid = u.unid
        JOIN participates p ON u.unid = p.unid
        JOIN mission m ON p.mid = m.mid
        GROUP BY s.sid
    LOOP
        IF rec.missions >= 5 THEN
            UPDATE soldiers SET rank = 'admiral'
WHERE sid = rec.soldier_id;
        ELSIF rec.missions >= 3 THEN
            UPDATE soldiers SET rank = 'captain'
WHERE sid = rec.soldier_id;
        ELSE
            UPDATE soldiers SET rank = 'sailor' WHERE
sid = rec.soldier_id;
        END IF;
    END LOOP;
END;
$$;
```

הפרוצדורה נוצרה בהצלחה:



הפרוצדורה רצה בהצלחה:

```
CALL pr_promote_commanders();
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 928 msec.
```

פרוצדורה 2 - הפרוצדורה pr_auto_assign_crewmate משייכת חייל לצוות תחת מפקד קיים עם תפקיד מוגדר, וזורקת חריגה אם לא נמצא צוות למפקד.

```
-- Procedure that assigns a soldier to a crew under a specific commander with a specified role.
-- Throws an exception if no crew exists for the commander.

CREATE OR REPLACE PROCEDURE
pr_auto_assign_crewmate(soldier_id int, commander_id int, crew_type varchar)
LANGUAGE plpgsql
AS $$
DECLARE
    crew_id int;
BEGIN
    -- Find crew assigned to the commander
    SELECT cr.cid INTO crew_id
    FROM crew cr
    WHERE cr.cid = commander_id
    LIMIT 1;

    -- Raise error if no crew found
    IF crew_id IS NULL THEN
        RAISE EXCEPTION 'No crew exists for commander ID %', commander_id;
    END IF;

    -- Assign soldier to the crew
    INSERT INTO crewmate (crid, cid, type)
```

```sql
    VALUES (soldier_id, crew_id, crew_type);

    RAISE NOTICE 'Soldier % assigned to crew % under commander % as %',
        soldier_id, crew_id, commander_id, crew_type;
END;
$$;


CALL pr_auto_assign_crewmate(6666, 99, 'Gunner');
```

הפרוצדורה נוצרה בהצלחה:

```
Procedures (2)
    pr_auto_assign_crewmate(IN soldier_id integer, IN com
    pr_promote_commanders
```

הפרוצדורה רצה בהצלחה:

```
CALL pr_auto_assign_crewmate(200, 99, 'Gunner');
```

Data Output | Messages | Notifications

```
NOTICE:  Soldier 200 assigned to crew 99 under commander 99 as Gunner
CALL
```

תוכניות ראשיות

תכנית ראשית 1:

- מנסה לשייך חייל לצוות תחת מפקד מסוים עם תפקיד מוגדר באמצעות pr_auto_assign_crewmate.
- במקרה של שגיאה בשיוך, מציגה הודעת כשלון.
- מציגה את פרטי כל החיילים בצוותים של המפקד, כולל שם היחידה, מזהה הטנק והתפקיד בצוות, באמצעות fn_soldier_unit_summary.

```
-- This block tries to assign a soldier as a crewmate under a commander with a specific role,
-- then prints a summary of all soldiers under that commander.
DO $$
DECLARE
    sid INT := 502;
    cid INT := 301;
    role TEXT := 'engineer';
    rec RECORD;
BEGIN
    BEGIN
        CALL pr_auto_assign_crewmate(sid, cid, role);
    EXCEPTION
        WHEN OTHERS THEN
            RAISE NOTICE 'Failed to assign soldier % to commander % as %: %', sid, cid, role, SQLERRM;
    END;

    RAISE NOTICE 'Displaying crew members under commander %:', cid;

    FOR rec IN
        SELECT * FROM fn_soldier_unit_summary(cid)
    LOOP
        RAISE NOTICE 'Soldier % assigned to unit %, tank % as %',
            rec.soldier_id, rec.unit_name, rec.tank_id, rec.crew_type;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

הoutput של הרצת התוכנית:

```
NOTICE:  Displaying crew members under commander 301:
NOTICE:  Soldier 1831 assigned to unit  Tzavim, tank 301 as Loader
NOTICE:  Soldier 1670 assigned to unit  Tzavim, tank 301 as Gunner
NOTICE:  Soldier 154 assigned to unit  Tzavim, tank 301 as <NULL>
NOTICE:  Soldier 342 assigned to unit  Tzavim, tank 301 as <NULL>
DO
```

תכנית ראשית 2:

- מפיקה דוח ביצועים של כלל המפקדים באמצעות הפונקציה fn_commander_performance שמחזירה מצביע. (refcursor)
- מציגה לכל מפקד את מספר הטנקים, מספר המשימות ורמת הניסיון בהתאם למספר המשימות שביצע.
- מסווגת כל מפקד לרמות ניסיון: מנוסה, בינוני או חדש.
- מטפלת בשגיאות ומציגה הודעה במקרה של כשלון כללי.

```
-- This block fetches commander performance data via
a refcursor
-- and logs each commander's stats and experience
level.
DO $$
DECLARE
    ref refcursor;
    rec RECORD;
BEGIN
    -- Open the cursor returned by the function
    ref := fn_commander_performance();

    LOOP
        FETCH ref INTO rec;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Commander ID: %, # Tanks: %, #
Missions: %, Level: %',
            rec.commander_id, rec.num_tanks,
rec.num_missions, rec.experience_level;

        IF rec.experience_level = 'Experienced' THEN
```

```
            RAISE NOTICE '>> Commander % is highly
experienced.', rec.commander_id;
        ELSIF rec.experience_level = 'Intermediate'
THEN
            RAISE NOTICE '>> Commander % has moderate
experience.', rec.commander_id;
        ELSE
            RAISE NOTICE '>> Commander % is new or
inexperienced.', rec.commander_id;
        END IF;
    END LOOP;

    CLOSE ref;
EXCEPTION
    WHEN OTHERS THEN
        RAISE WARNING 'Error in commander performance
main program: %', SQLERRM;
END
$$ LANGUAGE plpgsql;
```

הoutput של הרצת התוכנית:

Query Query History

```
END
$$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

```
NOTICE:  Commander ID: 1, # Tanks: 1, # Missions: 2, Level: Intermediate
NOTICE:  >> Commander 1 has moderate experience.
NOTICE:  Commander ID: 2, # Tanks: 1, # Missions: 8, Level: Experienced
NOTICE:  >> Commander 2 is highly experienced.
NOTICE:  Commander ID: 3, # Tanks: 1, # Missions: 3, Level: Intermediate
NOTICE:  >> Commander 3 has moderate experience.
NOTICE:  Commander ID: 4, # Tanks: 1, # Missions: 3, Level: Intermediate
NOTICE:  >> Commander 4 has moderate experience.
NOTICE:  Commander ID: 5, # Tanks: 1, # Missions: 3, Level: Intermediate
NOTICE:  >> Commander 5 has moderate experience.
NOTICE:  Commander ID: 6, # Tanks: 1, # Missions: 8, Level: Experienced
NOTICE:  >> Commander 6 is highly experienced.
NOTICE:  Commander ID: 7, # Tanks: 1, # Missions: 4, Level: Intermediate
NOTICE:  >> Commander 7 has moderate experience.
NOTICE:  Commander ID: 8, # Tanks: 1, # Missions: 3, Level: Intermediate
```

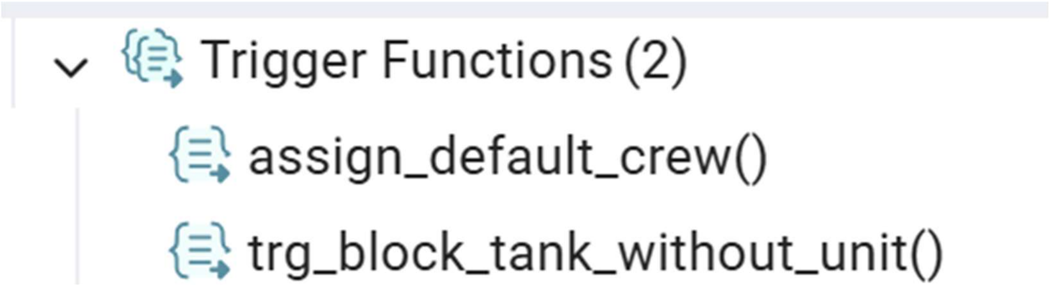
טריגרים

טריגר 1:

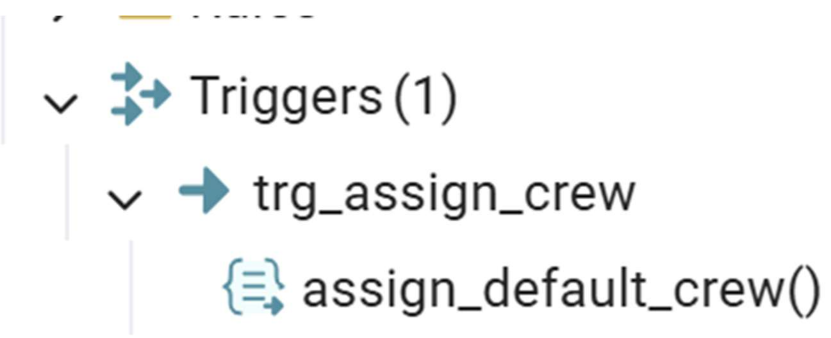
הטריגר trg_assign_crew משייך לחייל צוות ברירת מחדל לפני הכנסת הרשומה, אם לא צוין צוות בעת ההוספה לטבלת החיילים. (soldiers)

```
-- This trigger function assigns a default crew ID to
a soldier before insertion if none is provided.
CREATE OR REPLACE FUNCTION assign_default_crew()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.c_id IS NULL THEN
        SELECT c_id INTO NEW.c_id FROM crew ORDER BY
c_id LIMIT 1;
        RAISE NOTICE 'Assigned soldier % to default
crew %', NEW.sid, NEW.c_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_assign_crew
BEFORE INSERT ON soldiers
FOR EACH ROW EXECUTE FUNCTION assign_default_crew();
```

פונקציית הטריגר נוצרה בהצלחה:



הטריגר נוצר בהצלחה:

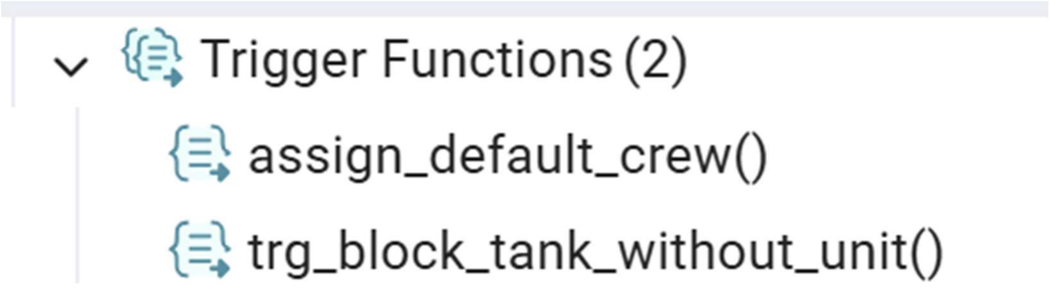


טריגר 2:

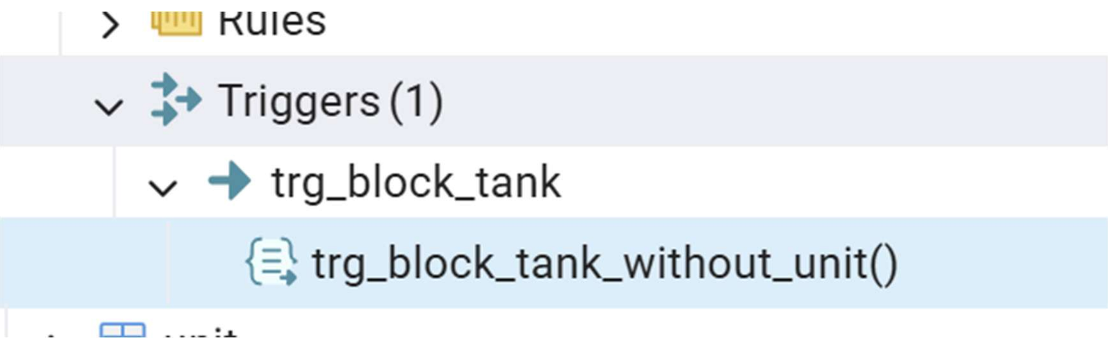
הטריגר trg_block_tank מונע הוספת טנק לטבלה tank אם לא צוין מזהה יחידה (unID) בעת ההכנסה.

```
-- This trigger prevents inserting a tank record
without an associated unit ID (unID).
CREATE OR REPLACE FUNCTION
trg_block_tank_without_unit()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.unID IS NULL THEN
        RAISE EXCEPTION 'Cannot insert Tank without
associated unit (unID)';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_block_tank
BEFORE INSERT ON tank
FOR EACH ROW
EXECUTE FUNCTION trg_block_tank_without_unit();
```

פונקציית הטריגר נוצרה בהצלחה:



הטריגר נוצר בהצלחה:



שלב 5 – ממשק גרפי

כלים

במסגרת הפרויקט פיתחנו אפליקציה גרפית לניהול נתונים הנשמרים בבסיס הנתונים (מסוג PostgreSQL), תוך שימוש בשפת Python ובספריות עזר שונות. מטרת האפליקציה היא לאפשר למשתמש קצה לבצע פעולות כמו צפייה, הוספה, עדכון ומחיקה של נתונים, הפעלת שאילתות פונקציות ופרוצדורות, בצורה נוחה וממשק גרפי ידידותי.

האפליקציה כוללת מספר מסכים עיקריים, דרכם ניתן לבצע מגוון פעולות ישירות מול בסיס הנתונים, מבלי צורך בהתעסקות עם שאילתות SQL בצורה ידנית.

לצורך פיתוח המערכת נעשה שימוש בכלים ובטכנולוגיות הבאים:

Python – שפת תכנות עוצמתית ופופולרית, אשר שימשה אותי כבסיס לפיתוח הלוגיקה של המערכת. פייתון מאפשרת חיבור מהיר ופשוט לבסיסי נתונים, ניהול ממשק גרפי, וביצוע פעולות על נתונים בצורה נוחה וברורה.

tkinter – ספריית GUI המובנית בתוך פייתון, באמצעותה יצרתי את ממשק המשתמש הגרפי של האפליקציה. הספרייה מאפשרת לבנות חלונות, כפתורים, שדות להזנת נתונים, הודעות קופצות ועוד. בחרתי ב-tkinter בזכות הפשטות, הזמינות והקלות שבה ניתן להקים איתה מסכים אינטראקטיביים.

psycopg2 – מודול פופולרי ב-Python המאפשר יצירת חיבור בין הקוד לבין בסיס נתוני PostgreSQL. באמצעות ספרייה זו, ניתן לשלוח שאילתות, לקבל תוצאות, ולהריץ פרוצדורות ופונקציות ישירות מתוך הקוד.

PostgreSQL (כולל pgAdmin) – מערכת ניהול בסיסי נתונים רלציונית (RDBMS) בקוד פתוח, המאפשרת אחסון נתונים בצורה מובנית ומסודרת. המערכת תומכת בקשרים בין טבלאות, פרוצדורות מורכבות, פונקציות ושאילתות מותאמות אישית. באמצעות pgAdmin, כלי הניהול הגרפי של PostgreSQL, בוצעה הקמה של בסיס הנתונים, הגדרת הטבלאות והקשרים, וכתיבת פונקציות ופרוצדורות המשמשות את האפליקציה.

מהלך העבודה

אחרי ארבעת השלבים האחרונים, בהם יצרתי את בסיס הנתונים, הכנסתי נתונים, וכתבתי שאילתות. ואחרי שלב האינטגרציה ושלב התכנות, התחלנו לעבוד על יצירת הממשק הגרפי. את פיתוח ממשק המשתמש ביצענו באמצעות שפת Python,
ובעזרת ספריית tkinter. בניתי מספר חלונות גרפיים ידידותיים למשתמש, דרכם ניתן לצפות בנתונים, לבצע חיפושים, להוסיף רשומות חדשות, לעדכן ולמחוק נתונים בצורה נוחה ואינטואיטיבית.
החלטתי להתמקד בחלק של חיל השריון.
ובפרט בטבלאות:

Participates
soldiers
commander
mission

הטבלאות הנ"ל קשורות אחת לשנייה, ולכן התמקדתי בהם.

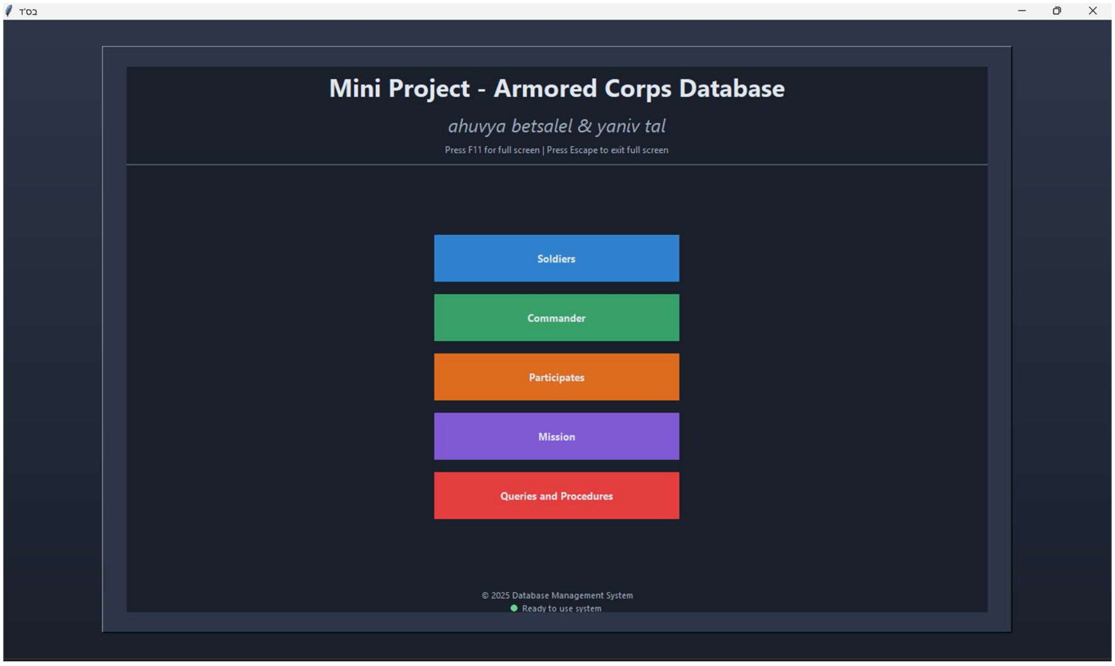
החיבור בין האפליקציה לבסיס הנתונים בוצע באמצעות מודול psycopg2,
שבאמצעותו שלחתי שאילתות SQL והפעלתי פרוצדורות ישירות מתוך הקוד.

יצרתי קובץ פייתון לכל אחד מהסכמות, בו דאגתי לתמיכה בפעולות הרצויות בנוסף לקובץ אחד שאחראי על הפעלת הפונקציות והשאילות וקובץ נוסף למסך הפתיחה ולהפעלת שאר המסכים.

התוצאה הייתה ממשק גרפי יפה וידידותי למשתמש שתומך במספר רב של פעולות ומאפשר ניהול קל של בסיס הנתונים.

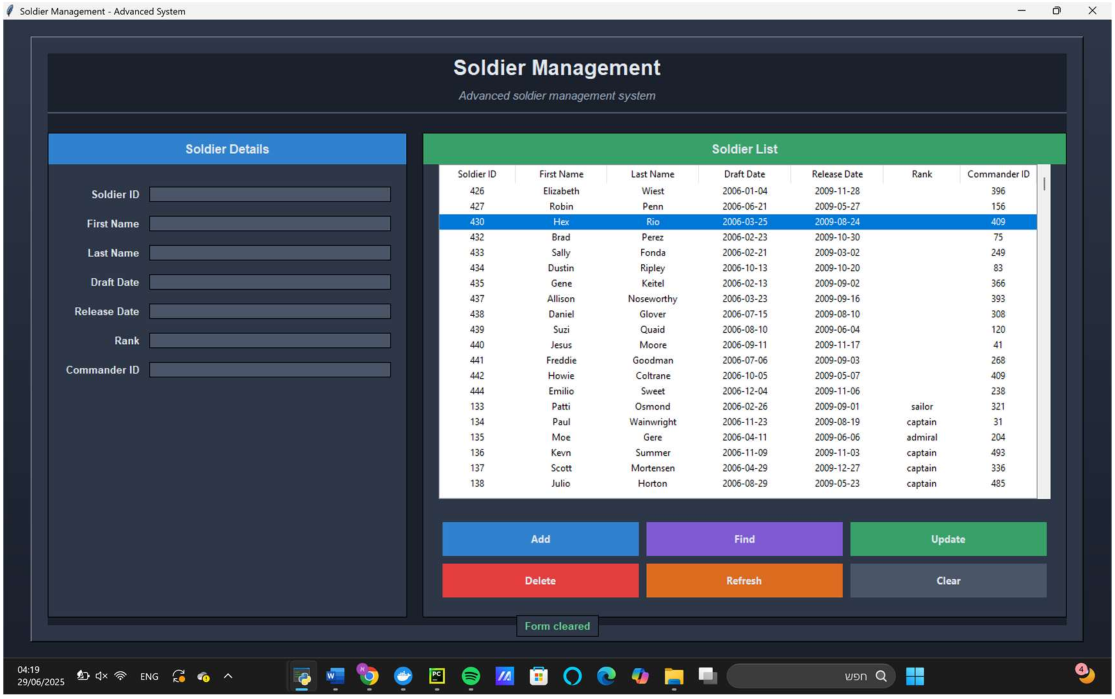
הוראות הפעלה של הממשק + צילומי מסך

אחרי הפעלת התוכנה ייפתח מסך הכניסה לאפליקציה.



כפי שניתן לראות, דרך מסך הכניסה ניתן להיכנס לכל אחד משאר חמשת המסכים.

מסך ניהול טבלת soldiers:

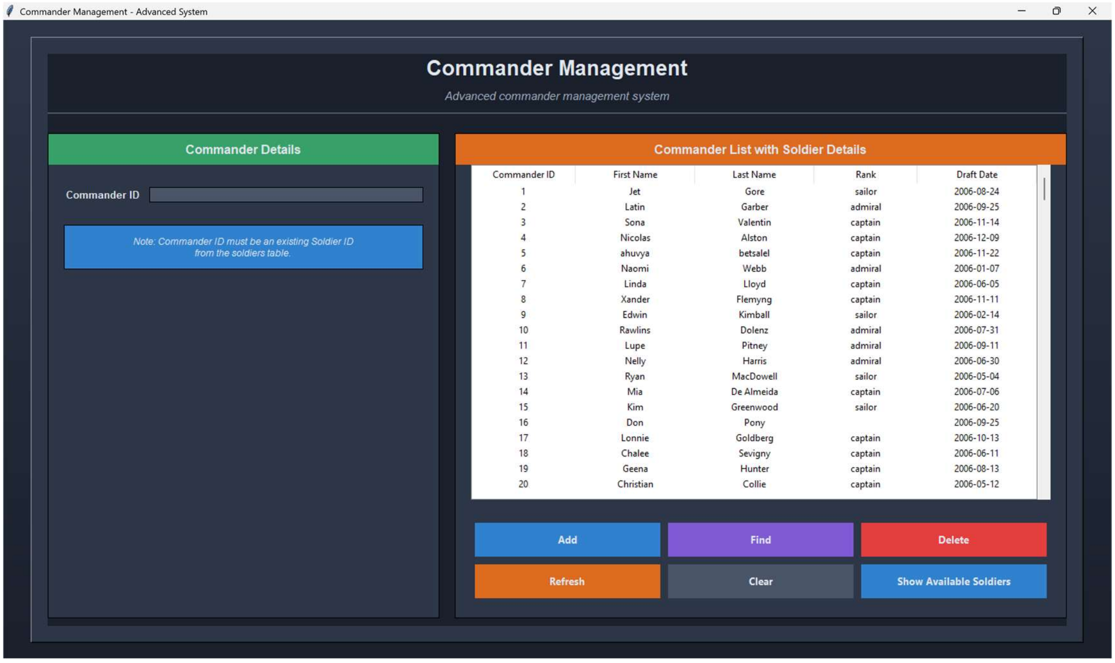


מסך זה תומך ב-6 פעולות.

Add – ניתן להוסיף רשומה לבסיס הנתונים.
find – חיפוש רשומה ספציפית.
update – עדכון רשומה מסוימת מהטבלה.
clear – מחיקת הפרטים משמאל לטבלה, ניקוי המסך...
refresh – כשמו כן הוא
delete – מחיקת רשומה מבסיס הנתונים.

כדי להפעיל אחד מ-6 הפעולות, נגלול בטבלה מימין ונלחץ על אחד השורות כדי לטעון את הפרטים לשדות משמאל, או שנכתוב באופן ידני בצד השמאלי של המסך את הפרטים הנדרשים. (ניתן להתחיל לכתוב את המפתח ולהשתמש ב-find כמובן.)

מסך ניהול טבלת commander:



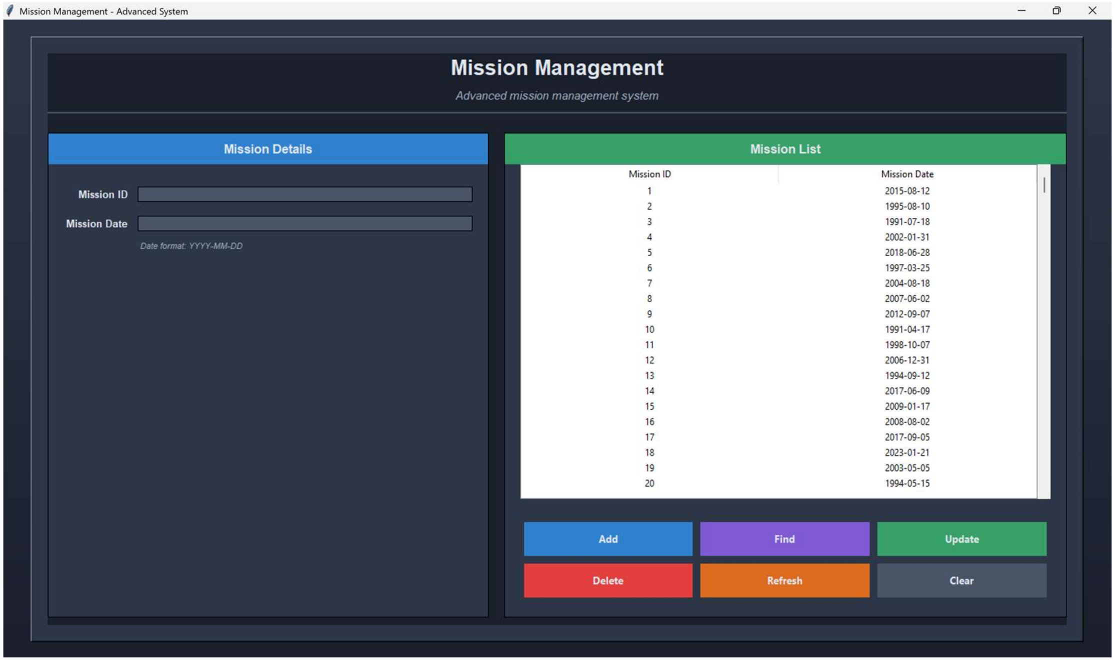
מסך זה תומך ב-6 פעולות.

Add – ניתן להוסיף רשומה לבסיס הנתונים.
find – חיפוש רשומה ספציפית.
show available soldiers – טעינת טבלת החיילים (מכיוון שכל מפקד חייב להיות חייל, זה נצרך לצורך הוספה וכו').
clear – מחיקת הפרטים משמאל לטבלה, ניקוי המסך...
refresh – כשמו כן הוא
delete – מחיקת רשומה מבסיס הנתונים.

כדי להפעיל אחד מ-6 הפעולות, נגלול בטבלה מימין ונלחץ על אחד השורות כדי לטעון את הפרטים לשדות משמאל, או שנכתוב באופן ידני בצד השמאלי של המסך

את הפרטים הנדרשים. (ניתן להתחיל לכתוב את המפתח ולהשתמש ב-find כמובן.)

מסך ניהול טבלת mission:

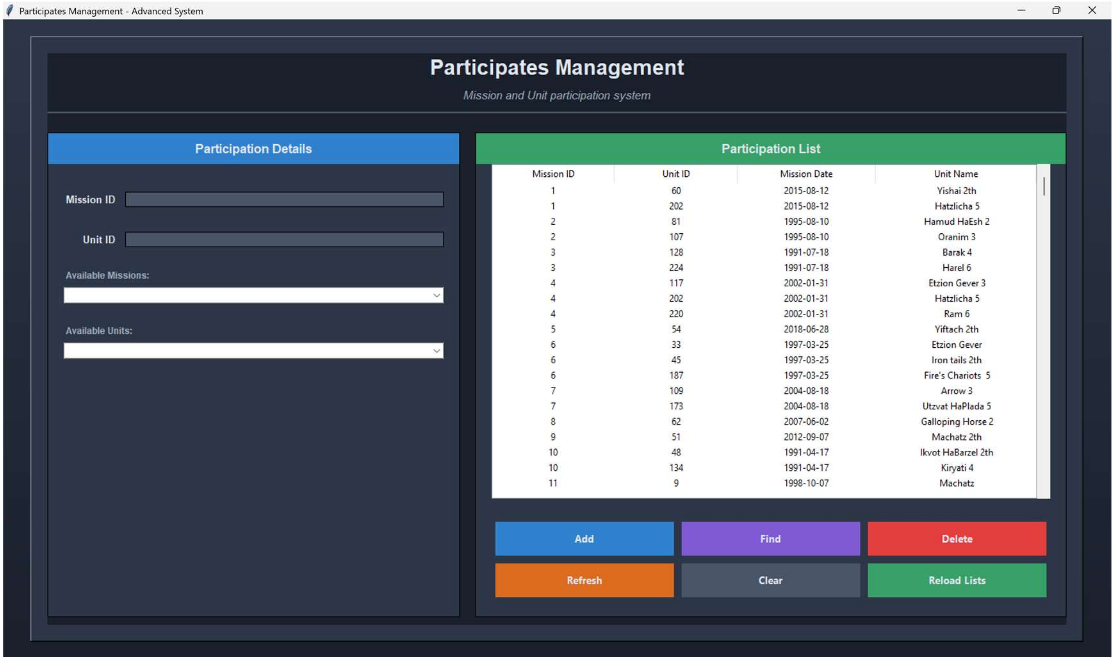


מסך זה תומך ב-6 פעולות.

Add – ניתן להוסיף רשומה לבסיס הנתונים.
find – חיפוש רשומה ספציפית.
update – עדכון רשומה מסוימת מהטבלה.
clear – מחיקת הפרטים משמאל לטבלה, ניקוי המסך...
refresh – כשמו כן הוא
delete – מחיקת רשומה מבסיס הנתונים.

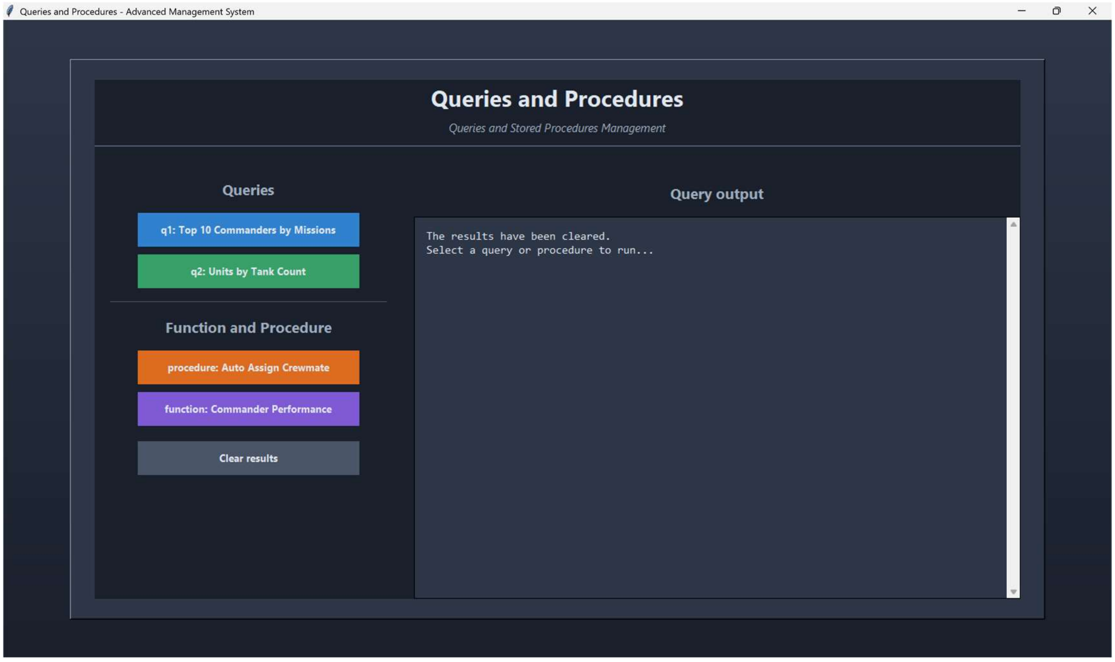
כדי להפעיל אחד מ-6 הפעולות, נגלול בטבלה מימין ונלחץ על אחד השורות כדי לטעון את הפרטים לשדות משמאל, או שנכתוב באופן ידני בצד השמאלי של המסך את הפרטים הנדרשים. (ניתן להתחיל לכתוב את המפתח ולהשתמש ב-find כמובן.)

מסך ניהול טבלת participates:



כמו בטבלאות שהצגנו, גם מסך זה תומך בפעולות הוספה, מציאה ע"י מפתח, מחיקה, ריענון, ניקוי מסך, וטעינה מחדש של הנתונים.
בנוסף כמו בטבלת commander הוא מאפשר טעינה של הטבלאות הרלוונטיות כדי לבצע בצורה נכונה את ההוספה וכו'.

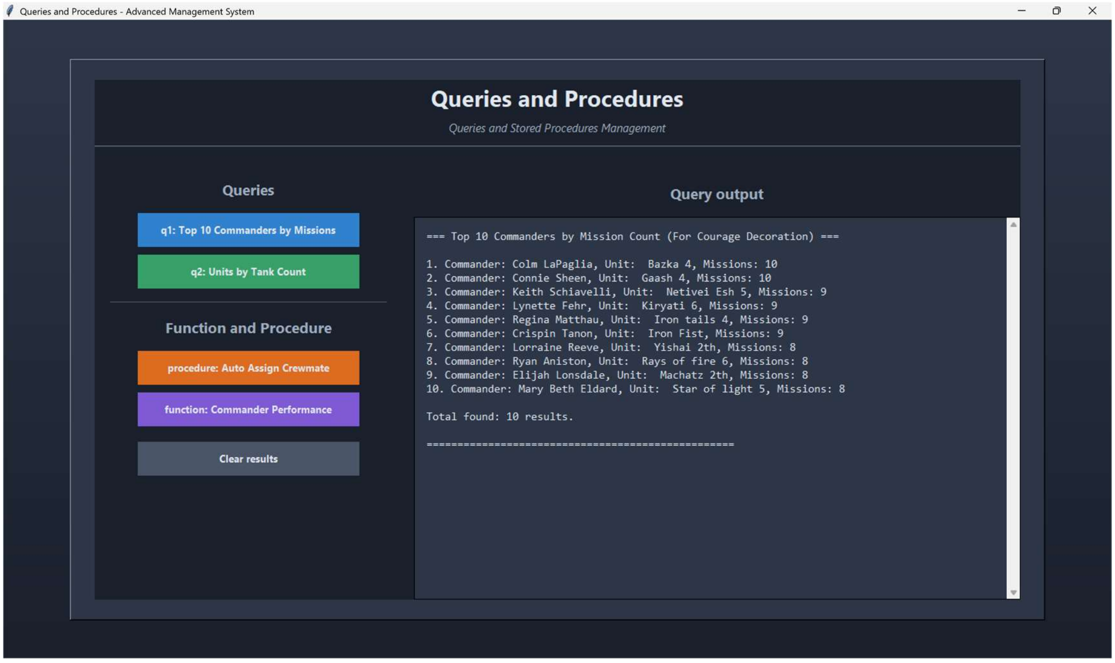
מסך הפעלת השאילתות הפרוצדורות והפונקציות:



כנדרש, הממשק הגרפי תומך בהפעלה של שאילתות מהשלב השני.

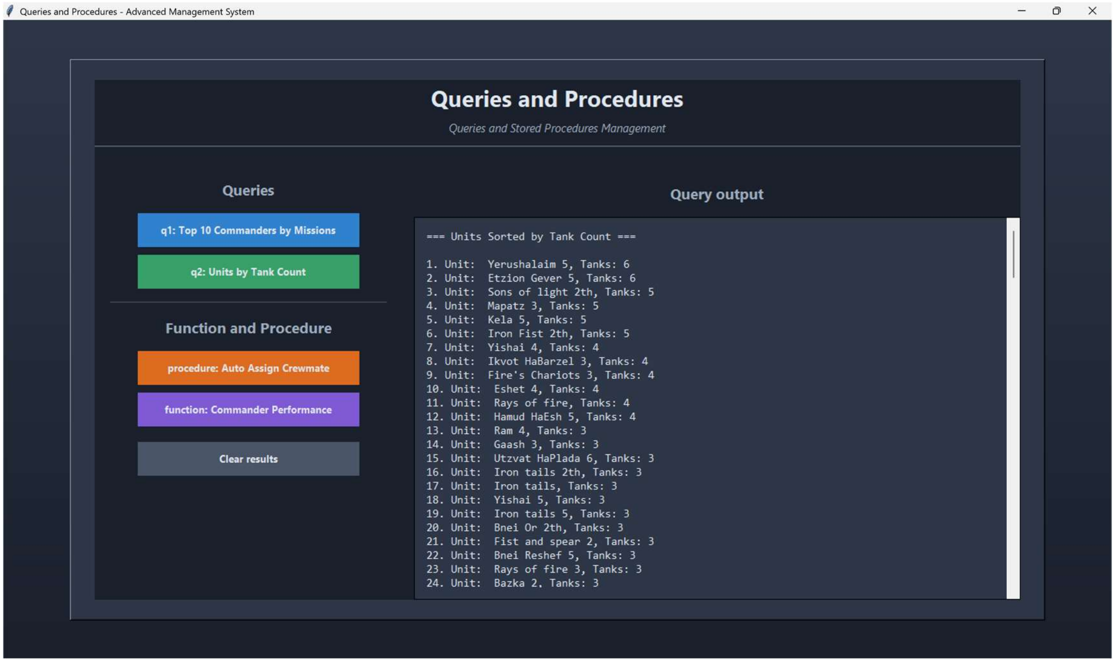
שאילתה 1 –

נפעיל אותה באמצעות לחיצה על הכפתור המתאים. את ההסבר עליה ניתן למצוא בדוח הפרויקט של שלב 2.



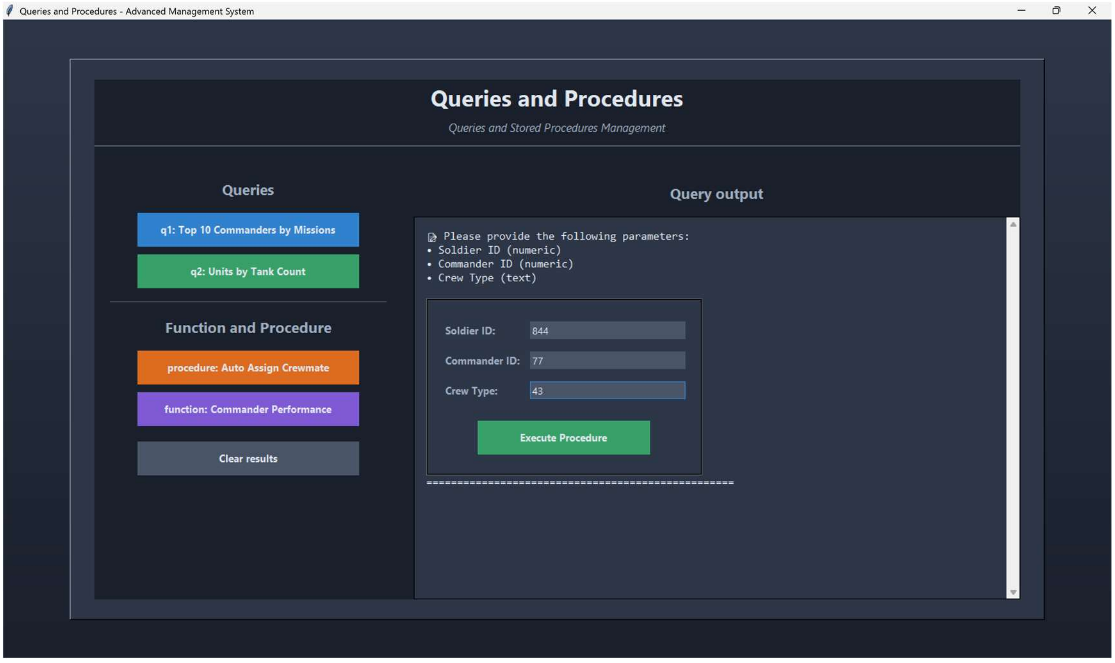
שאילתה 2 -

נפעיל אותה באמצעות לחיצה על הכפתור המתאים. את ההסבר עליה ניתן למצוא בדוח הפרויקט של שלב 2.

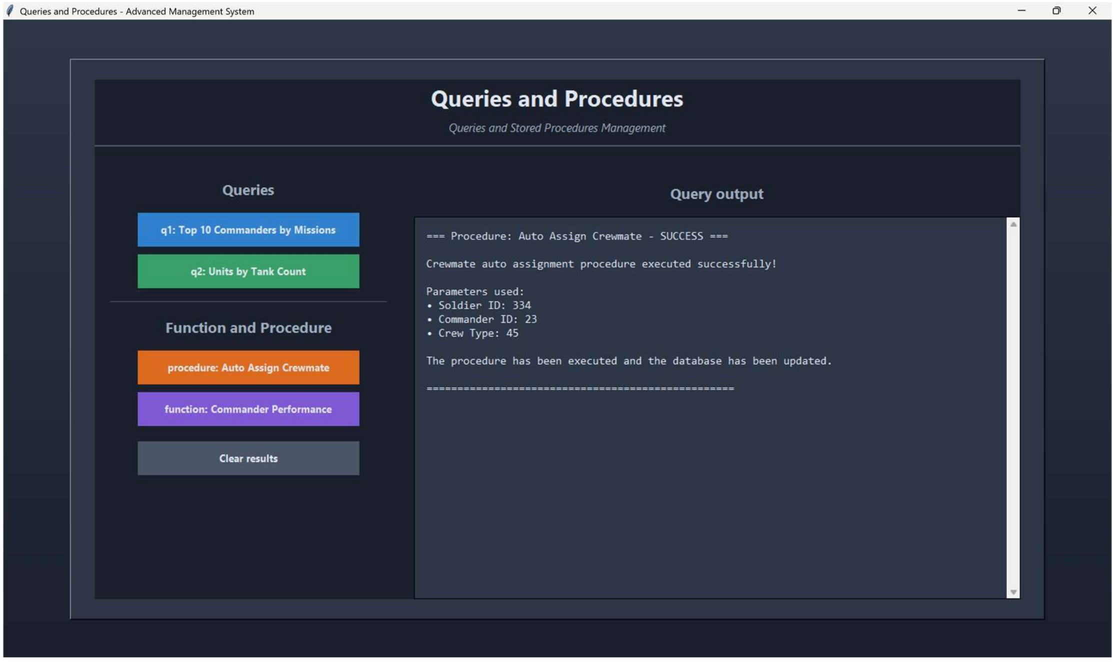
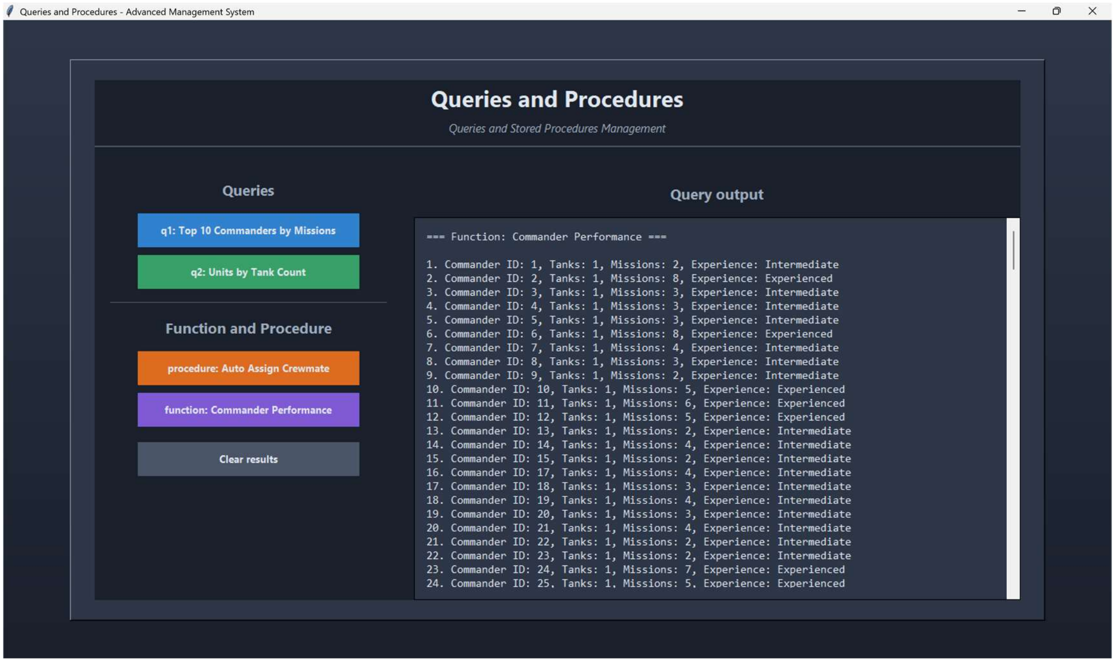


בנוסף הממשק תומך בהפעלה של פונקציה ופרוצדורה משלב 4.

פרוצדורה – לאחר הכנסת הנתונים הרלוונטיים, נלחץ על המקש execute procedure להפעלה של הפרוצדורה. (הסבר עליה ניתן למצוא בדוח הפרויקט שלב 4).



הפרוצדורה בוצעה בהצלחה:



פונקציה – לחיצה על המקש הסגול, תביא להפעלתה המוצלחת של הפונקציה (הסבר עליה ניתן למצוא בדוח הפרויקט שלב 4).